Construire la partie LabVIEW du projet Binairo

Recette de construction pour Louis Pédelaborde, Romeo Mugnier de Almeida et Raphaël Pical. Version du 11 novembre 2026, après l'addendum du 7 octobre et les corrections du professeur sur Ed du 29 au 31 octobre ; rangement en c, matlab et temp.

Ajouts du 11 novembre 2026 : inclure `Binairo_6x6_rect.v2.png` (exemple du professeur ajouté le 9 novembre) dans les images (§1), les contrôles de géométrie (G5.6), les résultats attendus (§18.1), les essais (V26) et les résultats conservés (§21.2). Recompiler `OCR.exe` sur la VM Windows avec l'optimisation activée (§1.1), puis effectuer toute la validation avec cet exécutable. Ces deux ajouts ne changent pas les sources C/MATLAB ni le câblage des VIs.

Modifications du 30 octobre 2026 (Ed #16, #28, #29, #30) : si vous avez déjà construit ces parties, reprenez §1.1 (recompiler OCR.exe, nouveaux résultats), §9.1 étape 1 et §9.3 (en-tête **hauteur puis largeur**), §11.1 étape 7 et §11.2 (case vide `-1` au lieu de `-2`). Les autres passages modifiés (§2.2, §3, §12.3, chapitre 20, annexes) ne changent que des explications. Deuxième série du même jour, après relecture : annexe G4 (case découpée **à l'intérieur des traits**, checkpoints 72×68 au lieu de 76×72), §15.2 (MATLAB lancé par la commande `matlab` **du PATH** quand le champ Executable MATLAB est vide, avec `-sd`), §18.3 (solution de repli pour ouvrir le PDF), §21.3 (points à documenter). Chaque passage modifié porte la mention *(modifié le 30.10)*.

Ce document explique comment construire les VIs, les raccorder au C et à MATLAB, puis vérifier le résultat. Il s'adresse à une personne qui débute dans LabVIEW. Les consignes sont prévues pour **LabVIEW 2025 Q3, menus anglais, sous Windows**, notamment dans la VM EPFL. Les noms anglais ci-dessous sont ceux à rechercher dans LabVIEW ; les noms de vos propres objets sont en français. Ne construisez pas un projet Real-Time, FPGA ou NXG.

Références prioritaires : `P00.PPI_Projet_soumission.26.2.pdf`, addendum révision 2 du **8 octobre 2026** *(modifié le 30.10)*, puis `P00.PPI_Projet.2026.2.pdf`, donnée du 7 septembre. L'addendum fait foi pour les noms, les erreurs C et les constantes (pages physiques 9–11). Les exercices précisent la construction. Les anciennes consignes Billard ne sont pas utilisées pour définir le contrat Binairo. Le critère MATLAB actuel est « utilisation minimale (~ absence) de boucles » ; les sources restent sans boucle explicite.

État réel : le C est compilé et testé sur macOS ; les nouveaux découpages ont été contrôlés avec cet exécutable à 88/90/90. Les VIs Windows et le rendu MATLAB natif restent à construire/exécuter sur la VM. Les résultats attendus ci-dessous sont des checkpoints à mesurer, pas des captures de VIs déjà réalisés. Les concepts utilisés et leurs références ME-213/CS-119a sont regroupés dans l'annexe H de cette recette. Aucun traitement d'image externe ni bibliothèque de résolution n'est nécessaire.

Règle OCR appliquée : la procédure détaillée de la donnée, page physique 18, impose `score >= seuil` pour les chiffres, puis le maximum de `score - seuil`. Elle prime ici sur le résumé « score brut » de sa page 10. Il existe donc une contradiction interne à la donnée, explicitement signalée ; le code suit désormais sa procédure détaillée. Depuis la correction du professeur (Ed #28 et message Semaine 7), les chiffres sont testés d'abord ; la case vide n'est testée qu'ensuite, avec `ratio de blancs > seuil vide`, et s'écrit `d:'-1', <ratio de blancs>%` (Ed #16). Si 0 et 1 atteignent leur seuil avec le même score ou la même marge, le C s'arrête avec le code 7 (Ed #7). *(modifié le 30.10)* Aucun connecteur principal définitif n'apparaît dans cette donnée : les connecteurs ci-dessous sont nos choix de construction, pas celui du Billard 2025.

Contenu du dépôt : `c/` contient uniquement les cinq sources/en-têtes C ; `matlab/` contient les sept fonctions MATLAB ; `temp/` contient cette recette en Markdown, HTML et PDF. La compilation, la création des projets, la construction des VIs et la production des résultats seront effectuées dans la VM Windows. Aucun exécutable ni fichier projet préconfiguré n'est fourni. Les pièces de remise seront préparées après les essais, suivant le chapitre 21.

Mode d’emploi de cette recette

1. Ouvrez ce document à côté de LabVIEW. La recette est en français ; les noms des menus et des primitives restent en anglais.
2. Suivez les chapitres 1 à 7. Au chapitre 8, suivez toute l’annexeG, puis revenez au chapitre 9. Continuez ensuite jusqu’au chapitre 23.
3. Pour chaque sous-VI : créez d’abord la face avant, attribuez le connecteur, construisez le diagramme, puis exécutez le checkpoint. Ne raccordez pas un sous-VI dont le checkpoint échoue.
4. Une flèche cassée signifie que le câblage n’est pas terminé. Une flèche entière ne prouve pas que le résultat est correct : comparez toujours aux résultats indiqués.
5. À la fin d’une séance, sauvegardez tous les fichiers. Copiez le dossier Execution dans votre sauvegarde privée persistante. Les modifications faites seulement dans la VM ne sont pas automatiquement présentes dans GitHub.
6. Dans la version HTML, utilisez le sommaire pour vous déplacer et les boutons Copier pour les chaînes exactes. Les cases « Étape terminée et contrôlée » sont une aide personnelle : ne les cochez qu’après un essai réussi. Elles ne constituent pas une validation automatique.

La chaîne construite sera :

```
PNG → LireImage → ComputeRowsCols
                  ↓
       pour chaque ligne, pour chaque colonne
       ComputeCellRect → Cell.bin → OCR.exe → CellValue.txt
                  ↓
       matrice de 0 / 1 / NaN → solve.m
                  ↓
       lanceur MATLAB → fonctions .m → PDF
```

Un seul dossier d’exécution, un seul nom de fichier par résultat, un seul fil d’erreur d’une étape à la suivante. Les VIs auxiliaires sont construits avant le programme principal ; vous n’avez pas d’architecture à choisir.

1 Préparer le dossier une seule fois

1. Démarrez la VM Windows. Ouvrez une session. Repérez un dossier personnel **persistant et inscriptible** : un fichier posé sur le bureau d'une VM peut être perdu lors d'une réinitialisation. Demandez au support EPFL si cette persistance n'est pas documentée. Ne changez pas l'installation logicielle du cours.
2. Dans l'Explorateur Windows, activez l'affichage des extensions de fichiers. Utilisez un dossier `Binairo` contenant les trois dossiers `c`, `matlab` et `temp`, comme dans le dépôt. Créez `temp/Execution`. Dans toute la suite, `Execution` désigne ce dossier ; `Images` et `Preuves` désignent `temp/Images` et `temp/Preuves`. Les chemins donnés dans les exemples de commandes ne sont pas à graver dans les VIs.
3. Copiez dans `Execution` **les cinq fichiers** du dossier `c` de la livraison : `main.c`, `reconnaissance.c`, `reconnaissance.h`, `parametres_ocr.h`, `FontRasterized_0_1.h`. Le dernier est la police originale du professeur, incluse dans le dépôt et dans son archive GitHub `Code > Download ZIP`. Ne créez pas une police de remplacement.
4. Copiez aussi les **sept fichiers** `.m` de la livraison dans `Execution` : `CheminPdfBinairo`, `CreerFigureBinairo`, `DispBinairo`, `DisplayBinairo`, `FormatGrilleValide`, `GrilleBinairoValide`, `SolveBinairo`. Gardez l'extension `.m` et la casse exacte. Le script généré ci-dessous appelle directement SolveBinairo et DisplayBinairo.
5. Le mode normal de cette recette lance MATLAB directement par System Exec. **Facultatif :** copiez `MP_LaunchMatlabScript4.vi` depuis Moodle uniquement si vous souhaitez utiliser ce lanceur alternatif ; conservez alors ailleurs son original.
6. Dans `Binairo/temp`, créez `Images` et `Preuves`. Copiez les PNG du cours dans `Images`, **jamais à la place d'un résultat**. En particulier : `Binairo_6x6.png`, **`Binairo_6x6_rect.v2.png`** (ajouté le 9 novembre), `Binairo_8x8.png`, `Binairo_4x4_Bad.png`, `Binaro_5x6_Bad.png` et **`BadBinairo.png`**. Ce dernier est volontairement un document Word renommé : gardez-le tel quel pour le test d'erreur. Le nom `Binaro_5x6_Bad.png` n'est pas à corriger.
7. Dans la VM, lancez `winver` et notez la version Windows réellement affichée dans `Preuves/version.txt` (Windows 11 seulement si confirmé). Ouvrez LabVIEW : `Help > About LabVIEW`, puis notez version et architecture. Faites de même avec MATLAB (`version` dans sa fenêtre de commande) et le compilateur (`gcc --version` ou la bannière de `cl`). Ces notes décrivent votre installation, pas le programme à remettre.
8. Dans LabVIEW : `File > Create Project > Blank Project`, puis enregistrez `Binairo.lvproj` **dans `Execution`**. Tous les VIs et `.ctl` de cette recette sont enregistrés à cet endroit, sous `My Computer`. Ajoutez les fichiers avec clic droit `My Computer > Add > File` si nécessaire.
9. Une seule personne modifie les VIs à la fois. Ne faites pas tourner deux exemplaires du programme dans le même dossier : ils partageraient `Cell.bin`, `CellValue.txt` et `solve.m`.

Après préparation : tous les `.vi`, `.ctl`, `.m`, les sources C et l'exécutable seront côte à côte dans `Execution`. Les images peuvent rester dans `Images`. Ne créez pas de copie de travail supplémentaire à synchroniser à chaque changement.

1.1 Compiler le C dans Windows

Effectuez la compilation dans la VM Windows. Ouvrez le terminal du compilateur **fourni dans votre environnement de cours**. Placez-vous dans `Execution` avec `cd /d "chemin réel vers Execution"` dans l'invite Windows. Cette saisie manuelle n'est pas un chemin absolu intégré au code.

- Si `gcc --version` fonctionne, utilisez :

```
gcc -std=c11 -O2 -Wall -Wextra -Wpedantic -Werror -Wvla main.c reconnaissance.c -o OCR.exe
```

- Si le cours utilise Visual Studio, ouvrez son **Developer Command Prompt**, puis utilisez :

```
cl /nologo /std:c11 /O2 /W4 /TC main.c reconnaissance.c /Fe:OCR.exe
```

Optimisation : `-O2` (GCC) ou `/O2` (Visual Studio) demande au compilateur de produire un exécutable plus rapide à partir des mêmes sources. Cela est utile pour les grandes cellules, jusqu'à 1000×1000 pixels ; ce réglage ne change ni les formats ni les seuils. Aucun délai maximal du professeur n'est confirmé ici. Dans un IDE, activez le même niveau d'optimisation dans la configuration de compilation utilisée pour le rendu. Refaites les essais ci-dessous et toute la validation du chapitre 19 avec cet `OCR.exe` Windows ; conservez cette même version testée pour la remise.

N'exécutez qu'une des deux commandes, selon le compilateur réellement installé. Si aucun n'est disponible, conservez la capture de l'erreur et faites installer/activer l'outil prévu par le cours ; ne poursuivez pas les tests OCR avec un exécutable Mac renommé `.exe`. Ne remplacez pas des erreurs de compilation par des modifications improvisées du code.

Copiez temporairement `Cell0.bin`, `Cell1.bin`, `CellEmpty.bin` et `BadCell_1.bin` du cours dans `Execution`, puis exécutez successivement :

```
OCR.exe Cell0.bin 88 90 90
echo %ERRORLEVEL%
type CellValue.txt
OCR.exe Cell1.bin 88 90 90
echo %ERRORLEVEL%
type CellValue.txt
OCR.exe CellEmpty.bin 88 90 90
echo %ERRORLEVEL%
type CellValue.txt
```

Résultats attendus : code `0` à chaque fois ; respectivement `d:'0', 97.167969%`, `d:'1', 97.753906%`, `d:'-1', 88.166545%`, chacun suivi d'une fin de ligne (Moodle indique 88.166542 % pour la case vide : l'écart sur la sixième décimale est admis par l'addendum). *(modifié le 30.10)* Le programme normal n'écrit rien sur stdout/stderr. Conservez une capture. Les fichiers du cours sont lus, jamais réécrits. Le résultat `CellValue.txt` est remplacé à chaque appel réussi. Lancez aussi `OCR.exe BadCell_1.bin 88 90 90` : code 3 et message de pixels manquants ; `CellValue.txt` est alors vidé (0 octet). Testez enfin les fichiers Cell rectangulaires (largeur ≠ hauteur) ajoutés sur Moodle : ils doivent être reconnus comme les cellules carrées. *(modifié le 30.10)* Dans chacune des trois positions de seuil, essayez 0, 100, 88.5, `nan` et `90junk` : code 2. Les valeurs 1 et 99 sont admises. Si vous travaillez depuis un IDE, créez le projet dans la VM avec les fichiers ci-dessus et les réglages du cours. Placez son OCR.exe compilé dans Execution. Conservez les

fichiers projet réellement créés dans la VM pour le rendu ; aucun projet préconfiguré n'est à récupérer dans le dépôt.

1.2 Tester MATLAB avant de construire le lanceur

Effectuez les contrôles ci-dessous directement dans MATLAB sur la VM. Conservez les observations et les PDF dans Preuves/Matlab, puis retirez les sorties de test de Execution avant l'intégration. Ces contrôles sont indépendants de LabVIEW ; les quatre options et le lancement réel seront vérifiés aux chapitres 18 et 19.

Après avoir copié les dernières fonctions .m, effectuez les trois contrôles simples ci-dessous dans MATLAB, avec Current Folder réglé sur Execution :

```
GrilleBinairoValide([0 1; 1 0])
GrilleBinairoValide([0 1; 0 1])
[G, ok] = SolveBinairo([0 NaN; NaN 0])
```

Attendez successivement TRUE, FALSE, puis `G = [0 1; 1 0]` avec `ok = TRUE`. La validation globale ne contient plus de parcours récursif ; les déductions conservent leur parcours équilibré pour respecter le critère « Pas de boucles » sans augmenter la profondeur de récursion. Les commentaires et interfaces du professeur restent inchangés. Ces contrôles sont à exécuter par vous, pas des résultats natifs déjà constatés.

Ouvrez MATLAB vous-même. Placez son **Current Folder** dans `Execution`, puis copiez ces lignes dans la Command Window :

```
B = [NaN 0 NaN NaN NaN NaN;
NaN NaN 0 NaN 0 NaN;
NaN NaN NaN NaN 1 0;
1 1 NaN NaN NaN NaN;
NaN 0 NaN 0 NaN NaN;
NaN NaN NaN NaN 0 NaN];
[S, ok] = SolveBinairo(B)
DisplayBinairo(B, S, 'Binairo_6x6.png', ok)
```

Attendez `ok = 1`, une grille complète sans modification des indices initiaux et `Binairo_6x6.pdf` dans `Execution`. Ouvrez le PDF : indices noirs gras, nouvelles valeurs bleues grasses, tous les traits, nom en haut à gauche et date/heure en haut à droite. La solution peut différer d'un exemple si une grille admet plusieurs solutions ; ses règles et indices doivent toujours être respectés.

Test d'échec logique :

```
B = [0 0 0 NaN; NaN NaN NaN NaN; NaN NaN NaN NaN; NaN NaN NaN NaN];
[S, ok] = SolveBinairo(B)
DisplayBinairo(B, S, 'Contradiction.png', ok)
```

Attendez `ok = 0` et un PDF avec `== Error ==`. Une grille insoluble n'est pas la même chose qu'une panne MATLAB. Une panne doit produire une erreur de lancement/script ; une grille insoluble doit tout de même produire le PDF demandé. Conservez les PDF dans `Preuves`, puis supprimez les copies de test d'`Execution` pour éviter de les confondre avec un futur résultat.

1.3 Compléter les contrôles MATLAB séparés

Vérifiez le format et les règles avec les expressions suivantes. Attendez FALSE pour chacune des six expressions :

```
FormatGrilleValide([])
FormatGrilleValide(zeros(2, 4))
FormatGrilleValide([0 2; 1 0])
GrilleBinairoValide(zeros(3, 3))
GrilleBinairoValide([0 0 0 NaN; NaN NaN NaN NaN; ...
    NaN NaN NaN NaN; NaN NaN NaN NaN])
GrilleBinairoValide([0 1 0 1; 0 1 0 1; ...
    NaN NaN NaN NaN; NaN NaN NaN NaN])
```

Testez l’affichage indépendamment du solveur, puis un nom contenant une apostrophe et un underscore :

```
DisplayBinairo([0 NaN; NaN 0], [0 1; 1 0], ...
    'Controle_affichage.png', true)
DisplayBinairo([0 NaN; NaN 0], [0 NaN; NaN 0], ...
    'L''exemple_test.png', true)
```

Inspectez les deux PDF : indices noirs gras, ajouts bleus gras dans le premier, cases vides dans le second, nom exact, date/heure et grille entière lisibles. Reprenez ensuite la matrice 8×8 du chapitre 18.1b dans B, appelez `[S, ok] = SolveBinairo(B)` et `DisplayBinairo(B, S, 'Binairo_8x8.png', ok)`. Attendez ok=TRUE, une grille complète valide et des indices inchangés. Pour le 6×6 comme le 8×8, ces trois expressions doivent donner respectivement TRUE, FALSE, TRUE :

```
GrilleBinairoValide(S)
any(isnan(S(:)))
isequal(S(~isnan(B)), B(~isnan(B)))
```

Avec la matrice 4×4_Bad du chapitre 18.1b, attendez ok=FALSE et un PDF portant `== Error ==`. Copiez chaque sortie dans Preuves/Matlab avant l’essai suivant. Ces vérifications ne remplacent pas les essais du solve.m généré par LabVIEW.

2 Les gestes LabVIEW à utiliser partout

2.1 Placer un objet et tirer un fil

1. `Ctrl+E` alterne entre **Front Panel** (les boutons/indicateurs) et **Block Diagram** (le programme). `Ctrl+S` enregistre.
2. Sur le diagramme, `Ctrl+Space` ouvre **Quick Drop**. Tapez le nom exact du nœud anglais indiqué, choisissez le résultat correspondant, appuyez sur Entrée, puis cliquez pour le déposer. N'utilisez pas un Express VI ni un objet IMAQ à la place d'une primitive homonyme.
3. Sur la face avant, clic droit dans le vide ouvre **Controls** ; utilisez sa recherche pour le type indiqué. Une **commande/control** fournit une entrée ; un **indicateur/indicator** affiche une sortie. Clic droit `Change to Control/Indicator` corrige la direction.
4. `Ctrl+H` ouvre **Context Help**. Survolez chaque borne avant de câbler : lisez son nom et son type. Quand le guide nomme une borne, trouvez-la par ce nom, pas par sa position supposée sur l'icône.
5. Avec la sélection automatique des outils, approchez une sortie jusqu'à obtenir la bobine, cliquez, puis cliquez sur l'entrée. Pour faire une branche, partez d'un fil existant. Un simple croisement n'est pas une connexion.
6. Donnez à chaque nœud l'identifiant indiqué (`N1` , `E1` , etc.) avec une étiquette libre juste à côté. Les identifiants recommencent dans chaque VI. `P1` désigne une borne de face avant, pas un nœud à rechercher.
7. Sur un nœud extensible, tirez son bord inférieur pour montrer le nombre exact d'entrées demandé. `x,y` désignent les deux opérandes affichés dans Context Help. Dans les tableaux, `sortie` désigne l'unique résultat du nœud ; `entrée 0` est la première entrée de haut en bas, même si l'aide la numérote 1.
8. Pour une constante numérique : clic droit `Representation` puis **I32**, **U32**, **U8** ou **DBL** comme indiqué. N'utilisez pas DBL pour des indices. Créez les constantes d'énumération **depuis la borne** avec `Create > Constant` , puis choisissez le libellé demandé.
9. Pour obtenir exactement le bon type d'une image, d'un chemin ou d'une erreur : clic droit sur la borne correspondante `Create > Control/Indicator/Constant` . Ne reconstruisez pas de mémoire les clusters NI.
10. N'utilisez pas `Run Continuously` . Cliquez une fois sur la flèche **Run** pour un essai. Si la flèche est cassée, cliquez dessus et traitez la première erreur de l'Error List ; gardez une capture si elle ne correspond pas à la recette.

2.2 Tableaux et clusters

- Un tableau 2D est indexé **ligne, colonne**. `Array Size` renvoie `[hauteur, largeur]` . Le fichier Cell.bin commence dans **le même ordre : hauteur, largeur** (Ed #29 et #30). *(modifié le 30.10)*
- Créez une coquille `Array` puis déposez son élément dedans. Clic droit sur l'afficheur d'index `Add Dimension` pour passer de 1D à 2D. Pour un tableau vide, `Data Operations > Empty Array` ; un tableau `[0]` n'est pas vide.
- Pour un cluster : placez `Cluster` , déposez ses éléments, puis clic droit sur la bordure `Reorder Controls in Cluster` . L'ordre est celui donné par la recette.

- Pour un type partagé `.ctl` : clic droit sur la commande cluster `Advanced > Customize`, choisissez **Type Def.**, enregistrez sous le nom prescrit, fermez et acceptez le remplacement. Déposez ensuite ce fichier depuis le projet. N'en recréez pas des copies indépendantes.
- `Build Array` : **Concatenate Inputs OFF** lorsqu'on assemble des scalaires ; **ON** lorsqu'on concatène des vecteurs en un seul vecteur. Le guide précise les exceptions.

Une **erreur claire** signifie ici : status=FALSE, code=0, source vide. Cela veut dire « aucune erreur ». Un cluster d'erreur n'est pas une chaîne de caractères ; créez-le depuis une borne error in/out.

2.3 Cases et boucles

Tous les sous-VIs sauf `ErreurSi.vi` ont une **Case Structure de garde**. Câblez le cluster `error in` à son sélecteur : les cases s'appellent **Error** et **No Error**. Dans Error, pas de lecture, écriture, lancement ou calcul d'index : renvoyez l'erreur inchangée et les sorties neutres indiquées. Dans No Error, placez les nœuds décrits.

Les tunnels de sortie d'une Case doivent être câblés dans **toutes** les cases. Laissez **Use Default If Unwired désactivé**. Une donnée traversant un bord devient un tunnel ; réutilisez ce même tunnel dans l'autre case. Les sorties d'une Case intérieure se raccordent aux sorties de la Case extérieure de même nom, puis aux indicateurs.

Une **garde supplémentaire** après une vérification signifie littéralement : poser une nouvelle Case, brancher l'erreur obtenue au sélecteur, placer les opérations suivantes dans No Error ; dans Error renvoyer les sorties neutres et cette erreur. Ce geste interdit notamment d'indexer un tableau ou de lancer un programme après validation échouée.

Pour chaque For Loop : le terminal `i` commence à 0. Un tunnel **auto-indexé** distribue un élément/une ligne à chaque tour ; un tunnel **non indexé** transmet l'objet entier. Clic droit sur le tunnel pour choisir. Un **Shift Register** se crée par clic droit sur le bord de la boucle `Add Shift Register` : câbler sa valeur initiale **à gauche, depuis l'extérieur** ; à l'intérieur lire la borne gauche et écrire la droite à chaque tour. Tous les registres de cette recette sont initialisés. Désactivez le parallélisme des boucles qui écrivent les fichiers ou appellent OCR.

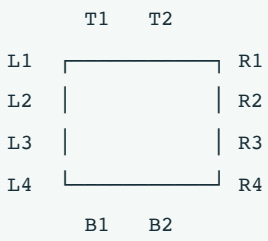
2.4 Chaînes et booléens

Normal Display conserve les caractères tapés tels quels dans la constante. **Backslash Codes Display** interprète `\n` comme LF et `\r` comme CR. **Format Into String interprète ensuite lui aussi les barres inverses de son format.** Pour transmettre les deux caractères `\n` à MATLAB, le format saisi en Normal Display doit donc contenir `\\n`. Suivez exactement le chapitre 13 ; ne confondez pas constante de format et résultat généré. Ne recopiez pas les accents typographiques d'un traitement de texte dans les commandes.

Les commandes booléennes d'options sont des **switches**, pas des boutons à verrouillage/latch. Défaut TRUE pour résoudre et afficher le PDF. Un contrôle numérique affiché arrondi conserve parfois davantage de décimales en mémoire : la recette ne suppose jamais le contraire.

2.5 Connecteurs des sous-VIs

Sur la face avant : clic droit sur l'icône en haut à droite `Show Connector`, puis `Patterns`. Choisissez le motif à **12 bornes, 4×2×2×4**. Ne le tournez pas. Convention : L1..L4 = côté gauche de haut en bas ; R1..R4 = côté droit de haut en bas ; T1,T2 = haut gauche à droite ; B1,B2 = bas gauche à droite.



Pour attribuer une borne : cliquez sur elle, puis sur sa commande/son indicateur. Les entrées sont à gauche et les sorties à droite selon les tables. Toutes les bornes non citées restent **non attribuées**. L4 et R4 servent à `error in` / `error out`. Pour vérifier, passez la souris avec Context Help sur une instance du sous-VI.

2.6 Sauvegarde et documentation de chaque VI

Après chaque checkpoint : `File > VI Properties > Documentation`. Copiez la phrase « But » du chapitre, puis ajoutez `Auteurs : Louis Pédelaborde, Romeo Mugnier de Almeida, Raphaël Pical.` Inscrivez les unités/indices et le comportement en cas d'erreur indiqués. Mettez le diagramme de gauche à droite sans fils passant sous les nœuds. Désactivez `Enable automatic error handling` dans `VI Properties > Execution` : les erreurs remontent au VI principal. Revenez à des commandes vides et une erreur claire, puis `Edit > Make Current Values Default`, `Ctrl+S`. Les valeurs temporaires de test ne sont pas les valeurs par défaut de remise.

3 Créer les types communs

Construisez les quatre `.ctl` suivants dans `Execution`. Tous les éléments sont des **commandes**, dans l'ordre indiqué.

Fichier	Éléments du cluster dans l'ordre	Valeurs par défaut
`SeuilsOCR.ctl`	`vide` DBL ; `zero` DBL ; `un` DBL	88 ; 90 ; 90
`OptionsBinairo.ctl`	`resoudre` booléen ; `afficherPDF` booléen	TRUE ; TRUE
`LignesGrille.ctl`	`debutX` , `finX` , `debutY` , `finY` , chacun tableau 1D I32	quatre tableaux vides
`RectangleCellule.ctl`	`gauche` , `haut` , `droite` , `bas` , chacun I32	quatre zéros

Les coordonnées droite/bas sont **exclusives** : une cellule `[gauche,droite)` a largeur `droite-gauche`. Les indices commencent à 0. Le cluster Rectangle natif des images NI utilise des I16 : ne le remplacez pas par `RectangleCellule.ctl`.

Les seuils officiels sont **88/90/90**, pour les exemples binaires et pour les cellules générées. Les trois valeurs doivent être des **entiers de 1 à 99**. Le type DBL permet de détecter et refuser une saisie fractionnaire au chapitre 10, au lieu de l'arrondir silencieusement dans un contrôle I32. Il ne rend pas les fractions admissibles.

Le découpage G4 coupe **juste à l'intérieur des traits** : la case ne contient aucun pixel des traits de la grille. Aucun pixel n'est ajouté ou recolorié. Sur les exemples fournis, un contrôle indépendant avec quatre seuils de conversion RGB (100, 128, 160, 200) donne 100 % de blanc pour les cases vides, 92,6–95,6 % de blanc pour les cases chiffrées, et des chiffres reconnus à 95,0–97,8 %. Les cases chiffrées dépassent donc 88 % de blanc : c'est pourquoi le C teste les chiffres avant la case vide (Ed #28, #30). L'ancien découpage, qui gardait deux pixels de trait, ne laissait que 1,5 point de marge au-dessus de 88 pour les cases vides (89,4–89,6 %) ; avec trois pixels, certaines tombaient sous 88. Ce contrôle est une approximation de la conversion native ; les checkpoints dans LabVIEW restent obligatoires. *(modifié le 30.10)*

Les scores de chiffres peuvent être décimaux, même si les seuils sont entiers. Le C accepte `score >= seuil`, choisit la plus grande marge `score - seuil` et s'arrête avec le code 7 si les deux chiffres sont admissibles avec le même score ou la même marge. Le score écrit est le score brut. Le test du vide n'intervient qu'après les chiffres et reste strict `>` ; une case vide s'écrit `-1` suivie du ratio de blancs. Les comportements de warning à décrire dans la documentation finale sont : arrêt sur pixels supplémentaires (code 3), arrêt si ni chiffre ni case vide n'est reconnu (code 5), sélection par marge si les deux chiffres sont admissibles, arrêt en cas d'égalité (code 7). Après une erreur, le C vide `CellValue.txt` lorsqu'il le peut. *(modifié le 30.10)* Aucun warning non fatal n'est écrit dans stderr ; le contrôleur du chapitre 12 est cohérent avec ce choix.

4 ErreurSi.vi

But : conserver une erreur existante ; sinon créer l'erreur demandée quand Condition est TRUE.

Face avant et connecteur : `Condition` booléen FALSE L1 ; `Code` I32 7001 L2 ; `Message` chaîne vide L3 ; `error in` L4 ; `error out` R4. Les codes 6101..6105 et 7001..7012 de la recette sont des codes choisis pour notre programme.

Posez N1 `Unbundle By Name` champ `status`, N2 `Bundle By Name` champs `status,code,source`, N3 `Select`, N4 `Select`, C1 booléen TRUE. Le cluster de base de N2 vient de `error in`.

Source	Destination
error in	N1.cluster et N2.input cluster
C1 TRUE	N2.status
Code	N2.code
Message	N2.source
Condition	N3.s
N2.cluster	N3.t
error in	N3.f et N4.t
N1.status	N4.s
N3.sortie	N4.f
N4.sortie	error out

Checkpoint : Condition FALSE transmet une erreur claire ; TRUE crée code/message choisis ; si `error in` contient déjà TRUE/123/`test`, la sortie reste exactement TRUE/123/`test`.

5 LireTexte.vi et EcrireTexte.vi

5.1 LireTexte.vi

But : lire tout un fichier texte sans masquer les erreurs et fermer le fichier même si la lecture échoue.

P1 `Chemin` Path L1 ; P2 `error in` L4 ; P3 `Texte` String R1 ; P4 `error out` R4. Garde extérieure : Error renvoie chaîne vide et P2.

Dans No Error : N1 `Open/Create/Replace File`, opération **open**, accès **read-only** ; N2 Case sur N1.error out. Branchez P1→N1.file path, P2→N1.error in. Prompt chaîne vide, disable buffering FALSE. Les autres sorties inutilisées restent libres.

N2.Error : Texte vide et N1.error out. N2.No Error : posez une `Flat Sequence Structure` **deux frames**. Frame 0 : N3 `Read from Text File`, `count` I32 **-1**, mode `Read Lines` OFF et `Convert EOL` OFF (clic droit). N1.refnum→N3.file ; N1.error out→N3.error in. N3.text traverse la séquence jusqu'à la sortie Texte. Frame 1 : N4 `Close File`, N5 `Merge Errors` à deux entrées. N1.refnum traverse la séquence→N4.refnum ; constante erreur claire FALSE/0/chaîne vide→N4.error in ; N3.error out→N5.entrée 0 ; N4.error out→N5.entrée 1 ; N5.error out→sortie erreur. Le passage à la frame 1 impose la fermeture **après** la lecture, même si la lecture a échoué.

Reliez sorties Texte/erreur de N2 aux tunnels correspondants de la garde extérieure, puis P3/P4. N2.Error et la garde Error ont bien leurs deux sorties câblées.

5.2 EcrireTexte.vi

But : remplacer/créer le fichier texte, écrire exactement la chaîne reçue, fermer et transmettre les erreurs.

P1 `Chemin` Path L1 ; P2 `Texte` String L2 ; P3 `error in` L4 ; P4 `error out` R4. Garde extérieure Error : P3→sortie. Dans No Error : même chaîne d'ouverture et de fermeture que ci-dessus, avec les différences **complètes** suivantes :

1. N1.operation = **replace or create**, access = **write-only** ; P3→N1.error in ; P1→file path.
2. N2.Error ne renvoie que N1.error out.
3. N2.No Error, frame 0 : N3 est **Write to Text File**, `Convert EOL` OFF ; N1.refnum→file ; P2→text ; N1.error out→error in. Aucun LF n'est ajouté automatiquement.
4. Frame 1 : N4 `Close File` avec erreur claire ; N5 `Merge Errors` reçoit N3.error out puis N4.error out, dans cet ordre ; résultat→N2→garde→P4. Refnum N1 traverse le tunnel de séquence.

Checkpoint : écrivez une chaîne `abc\n` saisie en Backslash Codes Display dans un fichier de test, relisez-la avec LireTexte : String Length doit donner 4. Recommencez avec `x\n` : longueur 2, pas de reste `c` de l'ancien fichier. Un dossier à la place du fichier doit produire une erreur, pas être remplacé.

6 SupprimerResultat.vi

But : effacer un ancien fichier généré avant une nouvelle exécution ; ne jamais supprimer un dossier.

P1 `Chemin` Path L1 ; P2 `error in` L4 ; P3 `error out` R4. Garde extérieure Error : P2→P3. Dans No Error : N1 `Check if File or Folder Exists`, P1→path, P2→error in. **N1.error out sélectionne une garde G1**, et doit être propagé.

1. G1.Error : N1.error out→sortie, sans suppression.
2. G1.No Error : N2 Case sélectionnée par N1.file or folder exists?. N2.FALSE transmet N1.error out : un fichier absent est un succès.
3. N2.TRUE : N3 `File/Directory Info`, P1→path et N1.error out→error in. N4 ErreurSi reçoit N3.directory→Condition, 7002→Code, `Le chemin du resultat est un dossier.` →Message, N3.error out→error in.
4. N5 Case sur N4.error out. Error : transmettre N4.error out. No Error : N6 `Delete`, P1→path, N4.error out→error in, FALSE→entire hierarchy, FALSE→confirm, chaîne vide→prompt. N6.error out→sortie.
5. Remonter chaque erreur N5→N2→G1→garde extérieure→P3. Ne remplacez aucun error out par une erreur claire.

Cette fonction ne reçoit que `CellValue.txt`, `solve.m`, ou le PDF calculé. Jamais le PNG, une source ou un dossier arbitraire. Checkpoint : absent = succès ; présent = supprimé ; dossier = erreur 7002 et dossier intact ; erreur entrante = conservée. Vérifiez aussi l'erreur d'accès sur un fichier protégé.

6.1 TexteCheminValide.vi — contrôler une chaîne sans expression régulière

But : refuser un chemin texte vide ou contenant un guillemet double, LF, CR ou NUL, avec les chaînes, boucles et comparaisons du cours.

Face avant : P1 `Texte` String L1, P2 `error in` L4, P3 `Valide` Boolean R1, P4 `error out` R4. Error : FALSE et P2. Dans No Error :

1. `String Length(P1)` donne L. For Loop avec N=L ; P1 traverse un tunnel non indexé.
2. Dans la boucle : `String Subset`, string=P1, offset=i, length=1. Sa sortie est le caractère courant.
3. Quatre `Equal?` comparent ce caractère aux quatre constantes : guillemet double saisi en Normal Display, et `\n`, `\r`, `\00` saisis en **Backslash Codes Display**. Les trois dernières constantes contiennent chacune un octet.
4. OR des quatre tests vers un tunnel de sortie auto-indexé : tableau des caractères interdits.
5. Hors boucle : `Or Array Elements` sur ce tableau ; OR avec `Equal To 0?(L)` ; NOT du résultat→Valide. P2→error out. Remonter les deux sorties.

Checkpoint : chaîne vide, guillemet double, retour à la ligne ou NUL donnent FALSE ; un chemin normal avec espaces, accent, apostrophe ou tiret donne TRUE. Avec erreur entrante 123 : FALSE et même erreur 123. Ce VI ne vérifie ni extension, ni existence : les appelants le font.

7 LireImage.vi

But : lire le PNG avec les fonctions natives LabVIEW, afficher l'image et fournir un tableau 2D TRUE=noir, FALSE=blanc. Aucune lecture d'image n'est déléguée à C, MATLAB ou Python.

7.1 Face avant et objets

P1 `PNG` Path L1 ; P2 `error in` L4 ; P3 `Bitmap` Boolean 2D R1 ; P4 `Image` **2D Picture** R2 ; P5 `Largeur` I32 R3 ; P6 `Hauteur` I32 T2 ; P7 `error out` R4. Créez Image depuis la sortie `new picture` de Draw Flattened Pixmap. Fond de l'indicateur blanc : `View > Tools Palette`, outil couleur, clic droit dans la zone de dessin, blanc.

La garde Error renvoie Bitmap vide 0×0, picture vide, largeur/hauteur 0, P2. Dans No Error, construisez les étapes suivantes dans cet ordre. Une garde après chaque validation empêche de convertir/indexer après erreur. Chacune de ses branches Error renvoie les mêmes quatre valeurs neutres et l'erreur de cette étape.

7.2 Lecture et dimensions

1. N1 `Read PNG File.vi` : P1→ `path to PNG file`, P2→ `error in`, constante U8 128→ `Transparency Thresh`. N1.error out sélectionne la garde G1 ; son image data entre dans G1.No Error. La sortie path de N1 est inutilisée.
2. Dans G1.No Error : U1 `Unbundle By Name` de image data, champ **Rectangle**. U2 `Unbundle By Name` du Rectangle, champs left/top/right/bottom. Posez quatre `To Long Integer` T1..T4 ; left→T1, top→T2, right→T3, bottom→T4.
3. N2 `Subtract` : T3−T1=Largeur. N3 `Subtract` : T4−T2=Hauteur. Faites six comparaisons : Largeur<32, Largeur>4096, Hauteur<32, Hauteur>4096, T1≠0, T2≠0. Chaque comparaison est un nœud séparé et a ses deux entrées câblées avec la constante I32 concernée.
4. Posez `Build Array` à six entrées scalaires booléennes, **Concatenate Inputs OFF**, puis `Or Array Elements`. Câblez les six résultats dans l'ordre ci-dessus→Build Array→OR. OR→ErreurSi.Condition ; code 6101 ; message `Dimensions PNG invalides.` ; N1.error out→error in. L'erreur obtenue sélectionne G2. Largeur, Hauteur, Rectangle original et image data traversent dans G2.No Error.

32..4096 est une borne de travail explicite ; l'image n'a pas besoin d'être carrée en pixels. La grille en cellules doit, elle, être carrée. Une image Word renommée `.png` doit échouer dès la lecture, pas être prise pour une image vide.

7.3 Conversion native en un bit

Dans G2.No Error, posez D1 `Draw Flattened Pixmap.vi`, D2 `Picture to Pixmap.vi`, D3 `Unflatten Pixmap.vi`, A1 `Array Size`, I1/I2 `Index Array` 1D.

Source	Destination
Picture vide créé depuis D1.picture	D1.picture

N1.image data	D1.image data
D1.new picture	D2.picture et tunnel Image final
Rectangle original U1	D2.rect
I32 1	D2.depth
U32 16777215, blanc RGB	D2.Background Color
D2.image data	D3.image data
D3.1-bit pixmap	A1.array et tunnel `brut`
A1.sizes	I1.array et I2.array
I32 0	I1.index
I32 1	I2.index

N'employez aucune autre sortie pixmap de D3. D1/D2/D3 n'ont pas à recevoir des bornes d'erreur inventées. D3.colors est conservé pour la polarité. Les autres sorties éventuelles de D2, ainsi que D3.top left/mask/autres profondeurs, sont inutilisées.

Vérifiez I1=Hauteur et I2=Largeur avec deux `Not Equal?` puis OR. Dépliez D2.image data avec `Unbundle By Name`, champ `image depth` : comparer à 1, ajouter ce résultat à OR. OR→ErreurSi.Condition, code 6101, message `Conversion image incoherente.`, erreur G2→error in. Garde G3 sur son résultat. G3.No Error reçoit brut, colors, Image, Largeur, Hauteur.

7.4 Normaliser la polarité

D3 renvoie des indices de palette : TRUE n'est pas à supposer noir sans vérification. Posez `Array Size` sur colors puis une Case numérique sur sa longueur, avec les cases **2**, **0**, **Default**. Ses sorties sont `noirVautTrue` Boolean et erreur.

Case 2 : I1/I2 `Index Array`, indices I32 0 et 1 sur colors. Pour chaque couleur U32, placez `Color to RGB` ; branchez color0/color1 à leur entrée couleur. Convertissez chacun des six résultats R/G/B en U32 avec `To Unsigned Long Integer` avant addition. Pour color0, deux Add calculent `(R0+G0)+B0` ; pour color1, deux Add calculent `(R1+G1)+B1`. `Less?` somme 1<somme 0→noirVautTrue. `Equal?` somme 1=somme 0→ErreurSi.Condition ; code 6105 ; message `Palette sans contraste.` ; erreur G3→error in ; résultat→erreur de Case.

Case 0 : placez deux constantes booléennes sur le diagramme, étiquetées `PolariteVerifiee` FALSE et `NoirVautTrue` TRUE. NOT(PolariteVerifiee)→ErreurSi.Condition ; code 6105 ; message `Verifier la polarite avec Binairo_6x6.png avant execution.` ; erreur G3→error in. NoirVautTrue→sortie booléenne ; ErreurSi.error out→sortie erreur. Ce réglage sera effectué une seule fois au checkpoint ci-dessous si la palette est vide.

Default : FALSE→noirVautTrue ; ErreurSi(TRUE,6105,`Palette 1 bit invalide.`,erreur G3)→sortie erreur.

Après cette Case : `NOT` sur le tableau brut entier ; `Select` avec s=noirVautTrue, t=brut, f=NOT(brut). La sortie de Select est le bitmap normalisé. Garde finale G4 sur l'erreur de palette : No Error renvoie ce Bitmap, Image, Largeur, Hauteur et erreur ; Error renvoie les valeurs neutres. Remontez ces cinq sorties dans G3→G2→G1→garde extérieure→indicateurs. Dans chaque branche Error extérieure, le cluster est celui qui a déclenché la branche, pas une erreur claire.

7.5 Checkpoint image et palette

1. Ouvrez LireImage seul, choisissez le vrai `Binairo_6x6.png`, lancez une fois. Si l'erreur est 6105 et demande de vérifier la polarité, mettez une sonde sur le fil `brut`, pas sur Bitmap neutralisé après erreur.
2. Dans le tableau de la sonde, regardez `[ligne0,colonne0]` et `[20,20]`. Dans ce fichier précis, le premier est noir, le second blanc.
3. Brut TRUE puis FALSE : `NoirVautTrue=TRUE`. Brut FALSE puis TRUE : `NoirVautTrue=FALSE`. Dans ces deux cas seulement, mettez `PolariteVerifiee=TRUE`, sauvegardez et relancez. Deux valeurs identiques : ne validez pas ; capturez le PNG choisi, la sonde et l'erreur pour correction.
4. Attendez largeur 454, hauteur 431, Bitmap[0,0]=TRUE, Bitmap[20,20]=FALSE et aucune erreur. La grille doit être visible sans inversion des noirs/blancs.
5. Avec `Binairo_8x8.png`, attendez largeur 610, hauteur 577 ; Bitmap[5,2]=TRUE et Bitmap[30,30]=FALSE. Les tailles sont celles des images reçues ; si Moodle a remplacé les fichiers, ne forcez pas ces nombres dans le diagramme.
6. Déplacez l'image dans un autre dossier, re-sélectionnez-la : elle doit encore être lue. La calibration n'est à refaire que si la version/environnement de conversion change.

8 Géométrie de la grille

Construisez maintenant, dans cet ordre : `EtendueNoire.vi`, `IntervallesNoirs.vi`, `ComputeCellRect.vi`, `ComputeRowsCols.vi`, en suivant l'**annexe G — Détecter la grille et découper ses cases** placée à la fin de cette recette. Revenez ensuite au chapitre 9. Les bornes de cellule sont **50 à 1000 pixels**, identiques au C et à l'addendum p. 10.

9 EcrireCellule.vi

But : écrire exactement le format attendu par `LireCellule` en C, sans dimensions supplémentaires : en-tête **hauteur puis largeur** (Ed #29 et #30), puis les pixels ligne par ligne. *(modifié le 30.10)*

P1 `Cellule` Boolean 2D L1 ; P2 `Chemin` Path L2 ; P3 `error in` L4 ; P4 `error out` R4. Garde Error transmet P3.

9.1 Valider puis convertir

Dans No Error : A1 `Array Size` sur P1 ; I1/I2 `Index Array` sur A1, indices 0/1 ; I1=hauteur, I2=largeur. Deux `In Range and Coerce` reçoivent hauteur/largeur avec lower=50 et upper=1000, **Include lower limit et Include upper limit TRUE**. AND des deux `in range?`, puis NOT→ErreurSi.Condition ; code 7003 ; message `Cellule hors des bornes 50 a 1000 pixels.` ; P3→error in. Garde G1 sur le résultat.

G1.No Error :

1. H1/H2 `To Unsigned Long Integer` sur **hauteur puis largeur**. H3 `Build Array` deux scalaires, Concatenate OFF : H1 (hauteur)→entrée 0, H2 (largeur)→entrée 1. Résultat = **U32 [hauteur,largeur]**. *(modifié le 30.10)*
2. F1 For Loop : P1 Cellule→tunnel auto-indexé, qui fournit une ligne Boolean 1D ; N non câblé. Dans F1, F2 For Loop : ligne→tunnel auto-indexé, qui fournit un booléen ; N non câblé. Dans F2, `Select` : s=pixel, t=U8 1, f=U8 0. Sortie auto-indexée de F2 = ligne U8. Cette ligne sort par tunnel auto-indexé de F1 = matrice U8.
3. M1 `Multiply` largeur×hauteur en I32. M2 `Reshape Array` sur la matrice U8, **une seule dimension de sortie**, M1→dimension size. Résultat = vecteur U8 dans l'ordre ligne par ligne. Ne transposez rien.

9.2 Écrire et fermer

N1 `Open/Create/Replace File` : operation **replace or create**, access **write-only**, disable buffering FALSE, prompt vide ; P2→file path ; erreur G1→error in. Garde G2 sur N1.error out. Error transmet cette erreur. No Error contient une Flat Sequence de deux frames.

Frame 0 : N2/N3 `Write to Binary File`. N1.refnum→N2.file ; H3→N2.data ; FALSE→N2.prepend array or string size? ; constante créée depuis byte order réglée **2 little-endian**→N2.byte order ; N1.error out→N2.error in. N2.refnum out→N3.file ; M2→N3.data ; FALSE→N3.prepend ; little-endian→N3.byte order ; N2.error out→N3.error in. Prompt vide pour les deux. Les sorties cancelled restent libres.

Frame 1 : N4 `Close File` reçoit refnum N1 et erreur claire ; N5 `Merge Errors` reçoit N3.error out puis N4.error out. Résultat→G2→G1→garde extérieure→P4. La séquence impose que les deux écritures finissent avant la fermeture. **Ne branchez pas le chemin à la seconde écriture : cela pourrait rouvrir/remplacer le fichier au lieu de continuer après l'en-tête.**

Le fichier contient exactement `8 + largeur*hauteur` octets. Le `prepend=FALSE` est volontaire : nous écrivons nous-mêmes l'en-tête U32 hauteur puis largeur, ce qui rend l'ordre visible dans le diagramme et indépendant des options de préfixe de LabVIEW. *(modifié le 30.10)*

9.3 Checkpoint asymétrique sans outil externe

Créez un VI temporaire `ControleBinaire.vi` dans un sous-dossier `Binairo/temp/Preuves/Essais`, pas dans les fichiers finaux à remettre.

1. `Initialize Array` : élément FALSE, dimension 0=55, dimension 1=60. `Replace Array Subset` : cette matrice→array, row=2, col=9, new element=TRUE.
2. Résultat→EcrireCellule.Cellule ; créez une commande Path pour un fichier temporaire `Asymetrique.bin`, erreur claire→error in, affichez error out.
3. Après son error out, posez `Read from Binary File` avec le **chemin** du fichier, constante de type U8 scalaire→data type, count I32 -1, byte order little-endian. La lecture U8 brute renvoie tous les octets ; ne lui donnez pas un type tableau qui attendrait une taille préfixée. Créez un indicateur tableau sur la sortie et un `Array Size`→indicateur.
4. Exécutez : taille 3308 ; huit premiers octets décimaux `[55,0,0,0,60,0,0,0]` (hauteur 55 puis largeur 60). *(modifié le 30.10)* À l'index **137**, valeur 1 ; les autres pixels 0. L'index 137 est `8 + 2*60 + 9`.
5. Testez ensuite une matrice 49×60 : erreur 7003. Revenez aux valeurs normales. Ce contrôle est de développement ; gardez-le dans Preuves, pas dans le code évalué si seuls les VIs de fonctionnement sont demandés.

10 CommandeOCR.vi

But : construire l'appel OCR avec le **chemin complet de Cell.bin**, comme demandé par la donnée p. 23, et les trois seuils. Les chemins sont calculés depuis le dossier du VI, jamais inscrits comme constantes personnelles.

P1 `Seuils` SeuilsOCR.ctl L1 ; P2 `error in` L4 ; P3 `Commande` String R1 ; P4 `error out` R4 ; **P5** `Dossier` **Path L2**, dossier absolu d'Execution. Garde Error : chaîne vide/P2.

Dans No Error : U1 `Unbundle By Name` donne vide,zero,un. Construisez les contrôles suivants, **pour chacun des trois DBL** :

1. `In Range and Coerce`, lower=DBL 1, upper=DBL 99, limites incluses TRUE. Garder seulement `in range?`, jamais `coerced x`.
2. `To Long Integer` convertit le DBL en I32. `Equal?` compare le DBL original à cet I32 (conversion numérique sur le comparateur). Une fraction n'est jamais égale à son arrondi.
3. AND de `in range?` et `Equal?`. Les trois AND→Build Array→And Array Elements→NOT→ErreurSi.Condition. Code 7004, message `Les trois seuils doivent etre des entiers de 1 a 99.`, P2→error in.
4. Garde G1 sur ce résultat. Dans Error, commande vide et erreur. **Les trois I32 du point 2 ne sont employés qu'en No Error**, après validation. NaN, Inf, 0,100 et les fractions sont refusés.

Dans G1.No Error, placer et câbler :

Objet	Entrées	Sortie
B1 Build Path	P5→base path ; chaîne `OCR.exe` →name	chemin absolu OCR
B2 Build Path	P5→base path ; chaîne `Cell.bin` →name	chemin absolu cellule
S1 Path To String	B1.path	chaîne exécutable
S2 Path To String	B2.path	chaîne cellule
F1 Format Into String	format ci-dessous ; initial string vide ; erreur G1→error in	commande et erreur

F1 reçoit **cinq arguments**, dans cet ordre : S1, S2, I32(vide), I32(zero), I32(un), issus de la validation ci-dessus. Saisir le format en **Normal Display** :

```
"%s" "%s" %d %d %d
```

F1.resulting string→Commande ; F1.error out→sortie erreur. G1.Error : chaîne vide et erreur G1. Remonter les deux tunnels vers la garde extérieure puis P3/P4.

System Exec lancera **l'exécutable directement** : ne pas ajouter `cmd /c` à cette commande. Les guillemets protègent les chemins contenant des espaces ; aucun shell n'a à développer un nom de dossier. P5

provient de Current VI's Path→Strip Path dans le principal, puis est transmis par LireCase. Working directory reste câblé séparément dans System Exec au même dossier.

Checkpoint : pour un dossier de test choisi `C:\Travail Binairo\Execution`, seuils 88/90/90, obtenir exactement :

```
"C:\Travail Binairo\Execution\OCR.exe" "C:\Travail Binairo\Execution\Cell.bin" 88 90 90
```

Ce chemin est un exemple de résultat : ne le copiez pas dans le diagramme. Les `%d` reçoivent des I32 et produisent des entiers sans séparateur décimal. Dans chacun des trois champs, testez 0,100,88.5,-1,101,NaN,Inf : erreur 7004 et commande vide. Testez 1 et 99 : commande construite. Les valeurs normales sont 88/90/90. Le checkpoint de LireCase et V19 vérifieront l'appel réel sous Windows, y compris dans un dossier avec espaces.

11 AnalyserResultatOCR.vi

But : transformer une réponse valide de C en `0`, `1` ou `NaN` pour MATLAB. Toute autre réponse est une erreur.

P1 `Texte` String L1 ; P2 `error in` L4 ; P3 `ValeurMatlab` String R1 ; P4 `Pourcentage` DBL R2 ; P5 `error out` R4. Sorties neutres : chaîne vide, DBL 0, erreur transmise.

11.1 Lire, reformater et comparer toute la ligne

Utilisez **Scan From String et Format Into String**, présentés au chapitre chaînes de LV2. Aucune expression régulière n'est nécessaire. Le reformatage impose les six décimales et les espaces exacts ; le scan seul serait trop permissif.

1. Dans la garde No Error : N1 `String Length(P1)` donne L. `Less?(L,2)` →ErreurSi(7005, `Ligne OCR trop courte.`,P2). Garde G1 sur son erreur : tous les String Subset suivants sont dans G1.No Error.
2. N2 `String Subset(P1,offset=L-1,length=1)` ; comparez à une constante `\n` saisie en **Backslash Codes Display**. NOT(Equal?)→ErreurSi(7005, `Fin de ligne OCR manquante.`,erreur G1). Garde G2.
3. Dans G2.No Error : N3 `String Subset(P1,offset=L-2,length=1)` ; comparez à `\r` saisi en Backslash Codes Display. `Select` : s=égalité, t=I32 2, f=I32 1. Soustraire cette valeur à L→longueurSansFin. N4 `String Subset(P1,offset=0,length=longueurSansFin)` → `ligne`.
4. N5 `Scan From String`, agrandi à deux sorties : input string=ligne, format Normal Display `%.;d:'%d', %f%%`, initial scan location=U32 0, default value1=I32 0, default value2=DBL 0, error in=erreur G2. Les sorties 1/2 sont `symbole` I32 et `score` DBL. Ajouter une garde G3 sur N5.error out. Une erreur native de scan est propagée : son code n'est pas nécessairement 7005.
5. Dans G3.No Error : N6 `Format Into String`, format Normal Display `%.;d:'%d', %.6f%%`, arg0=symbole, arg1=score, initial string vide, error in=N5.error out. N7 `Equal?` compare le résultat entier à `ligne`. `%.;` impose le point décimal dans les deux nœuds.
6. N8 `In Range and Coerce` reçoit score, lower=DBL 0, upper=DBL 100, deux limites incluses TRUE. N9 `String Length` sur N5.remaining string, puis `Equal To 0?`. AND(N7,N8.in range?,N9==0) puis NOT→ErreurSi(7005, `Format ou score OCR invalide.`,N6.error out). Garde G4 sur cette erreur. Les espaces superflus, un exposant, une virgule, une seconde ligne ou un nombre de décimales incorrect sont ainsi refusés.
7. G4.No Error : Case **numérique sur symbole**, cases 0,1,-1,Default *(modifié le 30.10)*, avec trois sorties ValeurMatlab/Pourcentage/erreur.

Case	ValeurMatlab	Pourcentage	Erreur
0	chaîne `0`	score	erreur G4
1	chaîne `1`	score	erreur G4
-1	chaîne `NaN`	score (ratio de blancs)	erreur G4

Default	chaîne vide	DBL 0	ErreurSi(TRUE,7005,`Symbole OCR invalide.`,erreur G4)

Dans -1, pas de garde supplémentaire : renvoyez directement NaN, le score lu et l'erreur G4 ; N8 a déjà vérifié que ce ratio est entre 0 et 100. *(modifié le 30.10)* Remonter les trois sorties via G4→G3→G2→G1→garde extérieure→P3/P4/P5. **Chaque Error renvoie chaîne vide, DBL 0 et l'erreur qui l'a sélectionné.**

11.2 Checkpoint du parseur

Saisissez les entrées ci-dessous en **Backslash Codes Display** pour fabriquer les fins de ligne, puis repassez en Normal Display si désiré.

Texte d'essai	Résultat
`d:'-1', 88.166542%\n`	`NaN`,88.166542,sans erreur
`d:'0', 97.167969%\n`	`0`,97.167969,sans erreur
`d:'1', 97.753906%\r\n`	`1`,97.753906,sans erreur
`d:'2', 90.000000%\n`	erreur 7005
`d:'0', 101.000000%\n`	erreur 7005
`d:'0', 90,000000%\n`	erreur de scan ou 7005, valeurs neutres
`d:'0', 90.000000%` sans LF	erreur 7005
ligne correcte puis seconde ligne	erreur 7005
chaîne vide	erreur 7005
`d:'0', 90.0%\n` ou `d:'0', 9e1%\n`	erreur 7005 après comparaison
`d:'-2', 0.000000%\n` (ancien format)	erreur 7005 (Default)

12 LireCase.vi

But : traiter exactement une cellule, dans l'ordre découpage → fichier → OCR → validation → lecture du résultat. Une erreur ne devient jamais une case vide.

12.1 Face avant

Commande ou indicateur	Type	Connecteur
Bitmap	Boolean 2D commande	L1
Lignes	LignesGrille.ctl commande	L2
Indices	cluster commande `ligne` I32 puis `colonne` I32	L3
error in	erreur commande	L4
Dossier	Path commande	T1
Seuils	SeuilsOCR.ctl commande	T2
ValeurMatlab	String indicateur	R1
Pourcentage	DBL indicateur	R2
Diagnostic	String indicateur	R3
error out	erreur indicateur	R4

Garde Error : chaîne vide,DBL 0,chaîne vide,error in. No Error contient U1 `Unbundle By Name` ligne/colonne, V1 `ComputeCellRect.vi`, U2 `Unbundle By Name` du Rectangle, deux Subtract et V2 `Array Subset` 2D.

12.2 Découper et écrire

1. Indices→U1.cluster ; U1.ligne/colonne→V1.ligne/colonne ; Lignes→V1.Lignes ; error in→V1.error in. V1.Rectangle→U2 ; U2.droite−gauche→largeur ; U2.bas−haut→hauteur.
2. Bitmap→V2.array ; U2.haut→V2.index0 ; hauteur→V2.length0 ; U2.gauche→V2.index1 ; largeur→V2.length1. Garde G1 sur V1.error out **avant toute écriture**. Les calculs purs peuvent être à gauche de G1, mais V2 peut aussi être placé dedans.
3. Dans G1.No Error : B1/B2 `Build Path` : Dossier→base path des deux ; chaîne `Cell.bin`→B1.name ; chaîne `CellValue.txt`→B2.name.
4. V3 `SupprimerResultat.vi` : B2→Chemin, V1.error out→error in. V4 `EcrireCellule.vi` : V2.subarray→Cellule, B1→Chemin, V3.error out→error in.

5. V5 `CommandeOCR.vi` : Seuils→Seuils, **Dossier→Dossier**, V4.error out→error in. V6 `System Exec.vi` : V5.Commande→command line, **Dossier→working directory**, TRUE→wait until completion?, TRUE→run minimized?, chaîne vide→standard input, constante créée depuis expected output size réglée 65536, V5.error out→error in.

12.3 Vérifier le programme avant de lire son fichier

Posez C1 `Not Equal?` return code≠I32 0 ; C2 `String Length` sur standard error ; C3 `Greater?` longueur>0 ; C4 OR(C1,C3). Posez F1 `Format Into String` avec **quatre arguments** ligne,colonne,return code,standard error, format saisi Backslash Codes Display :

```
Cellule ligne %d colonne %d : code OCR %d\nstderr : %s\n
```

Indices.ligne→arg0, colonne→arg1, V6.return code→arg2, V6.standard error→arg3 ; erreur claire→F1.error in ; initial string vide. F2 `Concatenate Strings` : F1.result, chaîne `stdout :` , V6.standard output ; résultat→Diagnostic et Message du V7 ErreurSi. C4→V7.Condition ; 7006→Code. `Merge Errors` à deux entrées reçoit V6.error out puis F1.error out→V7.error in.

V8 `LireTexte.vi` : B2→Chemin ; V7.error out→error in. V9 `AnalyserResultatOCR.vi` : V8.Texte→Texte ; V8.error out→error in. V9.ValeurMatlab/Pourcentage/error out→sorties correspondantes ; Diagnostic reste celui de F2 même en cas d'échec. Remontez G1 puis la garde extérieure vers les quatre indicateurs. G1.Error renvoie neutres et V1.error out.

Pourquoi supprimer le résultat avant chaque cellule : un ancien fichier ne doit pas survivre à un exécutable manquant ou défectueux. Le code de retour et le cluster System Exec sont deux contrôles distincts. Notre enveloppe LabVIEW considère stderr non vide comme diagnostic d'échec ; stdout est conservé pour inspection.

Checkpoint : pour la première cellule du 6×6, indices 0,0, les coordonnées sont gauche 6,haut 6,droite 78,bas 74, soit **72×68** et `Cell.bin` de **4904 octets**, dont les huit premiers sont `[68,0,0,0,72,0,0,0]`. Résultat `NaN`, avec `100.000000` comme pourcentage (case entièrement blanche). *(modifié le 30.10)* La case 0,1 doit donner `0`. Les fichiers Cell0.bin/Cell1.bin du cours ont un cadrage 74×74 différent : leurs dimensions ne sont pas le résultat attendu ici. Vérifiez aussi la case 2,4 (un `1`) : à 88/90/90 elle ne doit jamais devenir NaN.

13 GenererScript.vi

But : produire `solve.m` depuis la matrice réellement reconnue par les boucles LabVIEW. Aucun exemple prérempli ne remplace cette génération.

P1 `MatriceTexte` String L1 ; P2 `PNG` Path L2 ; P3 `Options` OptionsBinairo.ctl L3 ; P4 `error in` L4 ; P5 `Dossier` Path T1 ; P6 `Script` String R1 ; P7 `CheminScript` Path R2 ; P8 `error out` R4. Garde Error : chaîne vide, chemin vide, P4.

Dans No Error, posez les objets suivants et câblez exactement :

Objet	Entrées	Sortie utilisée
N1 Strip Path	P2→path	name, nom PNG seulement
N2 Search and Replace String	N1.name→string ; `'`→search ; `''`→replace ; I32 0→offset ; TRUE→replace all? ; mode regex OFF	chaîne avec apostrophes doublées
N3 Unbundle By Name	P3→cluster ; champs resoudre,afficherPDF	deux booléens
N4 Select	s=N3.resoudre ; t=chaîne `true` ; f=chaîne `false`	option résolution
N5 Select	s=N3.afficherPDF ; t=`true` ; f=`false`	option PDF
N6 Format Into String	format ci-dessous ; arg0=P1 ; arg1=N2 ; arg2=N4 ; arg3=N5 ; initial string vide ; P4→error in	texte script et erreur
N7 Build Path	base=P5 ; name=`solve.m`	chemin script
N8 EcrireTexte.vi	Chemin=N7 ; Texte=N6 ; error in=N6.error out	erreur finale

Le format N6 se saisit en **Normal Display**, avec de vrais retours à la ligne. Agrandissez la constante String et copiez tout le bloc suivant. Ne tapez pas les deux caractères `\n` à la place des retours à la ligne. Dans les deux chaînes MATLAB de fprintf, saisissez **deux barres inverses avant n**, soit `\\n` : Format Into String en consomme une. Le fichier généré en conservera une seule. Copiez ce bloc de format, pas le checkpoint de sortie :

```
%% Projet Binairo - ME-213
%% Auteurs : Louis Pedelaborde, Romeo Mugnier de Almeida, Raphael Pical
B = [
%s];
fichierSource = '%s';
disp(B);
if %s
    [S, ok] = SolveBinairo(B);
else
    S = B;
    ok = GrilleBinairoValide(B);
end
DisplayBinairo(B, S, fichierSource, ok);
if %s
    uiopen(CheminPdfBinairo(fichierSource), 1);
end
if ok
    fprintf('Binairo traite.\\n');
else
    fprintf('Grille contradictoire ou sans solution.\\n');
end
```

Les `%%` du format produisent chacun un seul `%` dans solve.m : les deux premières lignes sont les commentaires d'identification des auteurs.

N6.result→P6 ; N7.path→P7 ; N8.error out→P8, via la garde. Les `%s` ne sont pas des instructions MATLAB : ils sont remplacés par Format Into String. Les options deviennent **true/false**, jamais les nombres 0/1, pour produire des conditions MATLAB explicites. Les appels à SolveBinairo et DisplayBinairo figurent directement dans solve.m.

Checkpoint manuel : MatriceTexte contient deux lignes `0 1;\n1 0;\n`, PNG choisi `Grille.png`, options TRUE/FALSE. Ouvrez le fichier enregistré avec un éditeur de texte :

```
% Projet Binairo - ME-213
% Auteurs : Louis Pedelaborde, Romeo Mugnier de Almeida, Raphael Pical
B = [
0 1;
1 0;
];
fichierSource = 'Grille.png';
disp(B);
if true
    [S, ok] = SolveBinairo(B);
else
    S = B;
    ok = GrilleBinairoValide(B);
end
DisplayBinairo(B, S, fichierSource, ok);
if false
    uiopen(CheminPdfBinairo(fichierSource), 1);
end
if ok
    fprintf('Binairo traite.\n');
else
    fprintf('Grille contradictoire ou sans solution.\n');
end
```

Le nom affiché dans le PDF est le nom du PNG ; il n'est pas nécessaire de transmettre son chemin complet à MATLAB, qui n'a pas à relire l'image. Un nom `L'exemple.png` doit devenir `'L''exemple.png'`. Les noms fournis dans les essais du cours sont ASCII ; validez séparément l'encodage natif si vous employez des noms accentués sur Windows. Ne créez pas de protocole `MatlabStatus.txt` : les sources actuelles n'en écrivent pas.

14 Option facultative : inspecter le lanceur fourni

La donnée exige de lancer MATLAB avec le script généré et de détecter ses erreurs (p. 26 et 36). **Elle n'impose pas MP_LaunchMatlabScript4.vi.** Ce VI est proposé dans les fichiers utiles et employé par l'exercice. Le mode direct du chapitre 15.2 est donc le parcours normal de cette recette ; vous pouvez passer directement à ce chapitre sans inspecter le lanceur fourni. La page 6 de l'exercice LabVIEW 2 indique son rôle, sa co-localisation avec le script et une option interne de test ; elle ne donne pas les noms/types de toutes ses bornes. Cette inspection ne peut pas être remplacée par un connecteur inventé.

1. Ouvrez la copie d'Execution dans LabVIEW, sans modifier l'original Moodle. Montrez son Front Panel et son connector pane. Activez Context Help.
2. Cliquez chaque borne connectée du connector pane : la commande/indicateur correspondant doit se mettre en évidence. Notez dans `Preuves/lanceur.txt` pour **chaque** borne : position, nom exact, commande ou indicateur, type, valeur par défaut. Pour un numérique, clic droit Representation. Pour un cluster, notez ses éléments.
3. `Ctrl+E` : repérez l'option interne qui choisit un script de test. Vérifiez que la branche de fonctionnement utilise l'entrée de script. Ne laissez pas le test interne sélectionné.
4. Suivez le fil du nom/script : le VI attend-il `solve`, `solve.m`, ou un chemin complet ? Vérifiez par les nœuds de construction du chemin et par un essai manuel avec un petit script. Ne concluez pas à partir du seul libellé.
5. Repérez `System Exec.vi` ou le mécanisme de lancement. Son `wait until completion?` doit permettre de savoir quand le script est fini. Sous Windows, si MATLAB est lancé en ligne de commande, vérifiez le comportement synchrone effectif. Le simple fait que le VI retourne n'est pas une preuve.
6. Relevez la sortie error out, et les sorties éventuelles return code/stdout/stderr. Identifiez ce que le VI fait si MATLAB manque ou si le script appelle `error('Essai volontaire');`. Gardez les deux captures, succès et erreur.
7. Si ces essais donnent une erreur fiable et une attente complète, câblez le mode fourni du chapitre 15. S'il ne fournit pas ces garanties, **ne mettez pas un Wait de durée arbitraire** et ne marquez pas le lanceur comme validé. Le mode direct du chapitre 15 permet de tester le reste immédiatement ; gardez le mode direct pour la remise ; ne demandez une adaptation que si vous souhaitez cette option.
8. Si une nouvelle version du lanceur ou un `Binairo...ConPane.vi` est disponible sur Moodle, conservez aussi le fichier et son aide. Le connecteur de `Billard2025_ConPane.vi` ne doit pas être copié dans ce projet.

L'interface inconnue de ce VI facultatif ne bloque ni la construction ni la validation du mode direct. N'ajoutez pas une dépendance au VI fourni si vous ne l'utilisez pas.

15 LancerMatlab.vi

But : lancer `solve.m`, attendre sa fin, conserver le diagnostic et transmettre une erreur réelle en cas d'échec. Il n'effectue pas le calcul du Binairo lui-même.

15.1 Interface et préparation

P1 `Dossier` Path L1 ; P2 `ExecutableMatlab` Path L2, vide par défaut : vide signifie que Windows cherche la commande `matlab` dans son PATH, comme le lanceur du professeur *(modifié le 30.10)* ; P3 `LanceurFourni` booléen FALSE L3 ; P4 `error in` L4 ; P5 `Diagnostic` String R1 ; P6 `error out` R4. **FALSE est la valeur normale et le défaut enregistré.** TRUE est réservé à l'option du lanceur fourni si vous la construisez et la validez.

Garde Error : Diagnostic vide et P4. No Error : N1 `Build Path` base=P1, name= `solve.m` ; Case C1 sélectionnée par P3. C1.FALSE est le mode direct suivant ; C1.TRUE est le mode fourni décrit ensuite. Les deux renvoient Diagnostic et erreur aux sorties de la garde.

15.2 Mode direct entièrement déterminé

1. Dans C1.FALSE, N1b `Path To String` reçoit P1, puis N11 `TexteCheminValide.vi` (Texte=N1b, error in=P4). NOT(N11.Valide)→N12 ErreurSi.Condition ; code 7008 ; message `Dossier Execution vide ou contenant un guillemet ou un caractere de controle.` ; error in=N11.error out. Garde **G0** sur N12.error out ; G0.Error renvoie Diagnostic vide et cette erreur. *(modifié le 30.10)*
2. Dans G0.No Error : `String Length(N1b)`, puis `Subtract(longueur, I32 1)` vers l'offset d'un `String Subset` (string=N1b, length=I32 1). `Equal?` compare ce dernier caractère à une constante contenant une seule barre inverse `\`. `Concatenate Strings(N1b, .)` puis `Select` : s=égalité, t=concaténation, f=N1b. La sortie de Select est **dossierCommande**. Une barre inverse juste avant un guillemet fermant fausserait l'argument sous Windows ; `C:\.` désigne le même dossier que `C:\`.
3. N13 `Empty String/Path?` reçoit **P2** et sélectionne une Case **C2** à deux sorties, `Commande` (String) et `erreur`, câblées dans TRUE et dans FALSE.
4. **C2.TRUE, champ vide (cas normal, comme le lanceur du professeur) :** N6V `Format Into String`, initial string vide, error in=N12.error out, **un argument** dossierCommande, format A ci-dessous en Normal Display. N6V.result→Commande ; N6V.error out→erreur. Ne passez pas P2 vide dans TexteCheminValide.
5. **C2.FALSE, matlab.exe choisi :** N2 `Path To String(P2)` →N3 `TexteCheminValide.vi`, error in=N12.error out. `Strip Path(P2)` donne le nom ; `To Lower Case` puis `Equal?` avec la constante `matlab.exe` ; N4 AND de cette égalité et N3.Valide. NOT(N4)→N5 ErreurSi.Condition, code 7008, message `Choisir le fichier matlab.exe sans guillemet ni caractere de controle.`, error in=N3.error out. Garde G1. G1.No Error : N6E `Format Into String`, initial string vide, error in=N5.error out, **deux arguments** N2 puis dossierCommande, format B. G1.Error : chaîne vide et erreur G1. Les accents et espaces ne sont pas interdits.

Format A, champ vide :

```
matlab -wait -sd "%s" -batch "run('solve.m')"
```

Format B, matlab.exe choisi :

```
"%s" -wait -sd "%s" -batch "run('solve.m')"
```

Les deux commandes lancent directement l'exécutable, comme au chapitre 10 ; **ne pas ajouter cmd /c**. Avec un champ vide, Windows cherche `matlab` dans le PATH, exactement comme `MP_LaunchMatlabScript4.vi` du professeur. `-sd` fixe Execution comme dossier de départ de MATLAB, quelle que soit sa préférence Initial working folder : `run('solve.m')` et le PDF restent dans Execution. Aucun nom de PNG n'est injecté dans ces commandes.

6. Garde **G2** sur C2.erreur ; G2.Error renvoie Diagnostic vide et C2.erreur. Dans G2.No Error, N7 System Exec : C2.Commande→command line ; P1→working directory ; TRUE→wait until completion? ; TRUE→run minimized? ; chaîne vide→standard input ; constante expected output size=262144 créée depuis sa borne ; C2.erreur→error in. *(modifié le 30.10)*
7. N8 Format Into String trois arguments N7.return code, N7.standard output, N7.standard error ; erreur claire→error in, initial string vide. Format Backslash Codes Display : `MATLAB code %d\nstdout :\n%s\nstderr :\n%s\n`.
8. N9 Not Equal?(N7.return code,0)→N10 ErreurSi.Condition ; code 7008 ; N8.result→Message et Diagnostic. Merge Errors(N7.error out,N8.error out)→N10.error in ; N10.error out→sortie erreur. Remontez Diagnostic et erreur par G2→G0→sorties de C1.FALSE ; chaque case Error renvoie un diagnostic vide et l'erreur qui l'a sélectionnée. *(modifié le 30.10)*

MATLAB peut écrire un avertissement dans stderr sans échec : on conserve ce texte, mais le code de retour et le cluster d'erreur déterminent l'échec. Avant les essais, tapez `where matlab` dans l'invite `cmd` de la VM : si rien n'est trouvé, choisissez matlab.exe dans le champ ou faites ajouter MATLAB au PATH, puis relancez LabVIEW. N'enregistrez aucun chemin absolu comme valeur par défaut. *(modifié le 30.10)* L'option `-batch` doit être disponible dans MATLAB (R2019a ou plus récent). Vérifiez dans la VM ; si elle est plus ancienne, utilisez le lanceur du cours et retournez sa version au lieu d'inventer une commande différente.

15.3 Mode fourni et adaptation exacte à l'interface observée

Parcours normal, sans lanceur fourni : dans C1.TRUE, placez ErreurSi(TRUE,7009,`Option lanceur fourni non installee ; utiliser le mode direct.`,P4) ; câblez son error out à la sortie erreur et la même chaîne à Diagnostic. La case FALSE contient le lancement complet et constitue le fonctionnement normal. Cette branche TRUE signale une option indisponible, pas une fonction requise manquante. Gardez LanceurFourni=FALSE par défaut.

Seulement si vous installez l'option : remplacez ces deux sorties par l'intégration suivante et placez **le vrai MP_LaunchMatlabScript4.vi**. Utilisez la fiche du chapitre 14 pour appliquer cette table, uniquement aux bornes qui existent réellement :

Borne observée	Câblage
Entrée script de type Path	N1.path→cette borne
Entrée script String dont le diagramme attend le chemin complet	Path To String(N1.path)→cette borne

Entrée script String attendant le nom avec extension	constante `solve.m` →cette borne
Entrée script String attendant le nom sans extension	constante `solve` →cette borne
Entrée dossier de travail Path, si présente	P1→cette borne
Entrée exécutable Path, si présente	P2→cette borne
Entrée exécutable String, si présente	Path To String(P2)→cette borne
error in, si présente	P4→cette borne
error out, si présente et testée	cette borne→chaîne de contrôle d'erreur
Sélection de test interne exposée, si présente	valeur choisissant le script fourni, vérifiée au chapitre 14

Toutes les autres entrées doivent être identifiées et leur défaut confirmé par l'aide/diagramme : **une borne inconnue est une information à retourner, pas une invitation à deviner**. Si le VI n'a pas error in, il reste protégé par la garde extérieure de LancerMatlab. Si error out n'existe pas ou ne reflète pas une vraie erreur de script, ce mode n'est pas encore validé : conservez un ErreurSi(TRUE,7009, `Interface du lanceur fourni a verifier.` ,P4) et diagnostic identique jusqu'à correction fondée sur sa vraie interface. Cela empêche un faux succès.

Si return code/stdout/stderr existent, reprenez N8/N9/N10 du mode direct avec ces **sorties réelles**. S'il n'existe qu'un error out fiable, celui-ci devient la sortie erreur ; Diagnostic vaut une chaîne constante `Lanceur fourni termine ; verifier le PDF et error out.` . Le contrôle du fichier frais, au chapitre 16, reste obligatoire. Il ne remplace jamais une sortie d'erreur fiable.

Checkpoint pour les deux modes validés : script 2×2 de chapitre 13→fin d'exécution, PDF nouveau ; script `error('Essai volontaire');` →erreur visible et aucun succès annoncé. Renommez temporairement l'exécutable MATLAB sélectionné **dans la commande de test en choisissant un chemin inexistant**, sans toucher au fichier d'installation : erreur. Remettez ensuite le script normal par GenererScript.

16 VerifierPDF.vi

But : confirmer qu'après une exécution réussie de MATLAB, le fichier attendu existe et n'est pas vide. Le vieux PDF doit avoir été supprimé avant le lancement.

P1 `CheminPDF` Path L1 ; P2 `error in` L4 ; P3 `error out` R4. Garde Error transmet P2. No Error : N1 `File/Directory Info`, P1→path,P2→error in. N2 `Equal To 0?` sur la sortie size ; N3 OR(N1.directory,N2). N4 ErreurSi : Condition=N3, Code 7007, Message= `PDF absent, vide ou remplace par un dossier.`, error in=N1.error out ; sortie→P3. Le type de size est celui fourni par NI, sans conversion réductrice. Un fichier absent provoque l'erreur native de N1.

Cette fonction ne prouve pas la mise en page ni la validité logique. Une grille insoluble avec PDF `== Error ==` reste une production correcte. Le contrôle visuel et les exemples attendus viennent au chapitre 19.

17 BinairoSolver.vi le programme principal

17.1 Construire la face avant

Enregistrez `BinairoSolver.vi` dans `Execution`. Ajoutez ces objets ; textes et clusters exacts :

ID	Nom	Type et défaut	Borne
P1	Image PNG	Path commande, vide	L1
P2	Seuils	SeuilsOCR.ctl,88/90/90	L2
P3	Options	OptionsBinairo.ctl,TRUE/TRUE	L3
P4	error in	erreur commande claire	L4
P5	Executable MATLAB	Path commande, vide = commande `matlab` du PATH *(modifié le 30.10)*	T1
P6	Lanceur fourni	Boolean commande,FALSE par défaut	T2
P7	Image originale	2D Picture indicateur,vide	R1
P8	Matrice reconnue	String indicateur multiligne,vide	R2
P9	Diagnostic	String indicateur multiligne,vide	R3
P10	error out	erreur indicateur	R4
P11	Dossier execution	Path indicateur,vide	non connecté
P12	Taille grille	I32 indicateur,0	non connecté
P13	Bitmap	Boolean 2D indicateur,vide	non connecté
P14	Script genere	String indicateur multiligne,vide	non connecté
P15	PDF attendu	Path indicateur,vide	non connecté

B1/B2 non attribuées. Alignez P1..P6 à gauche, l'image à droite, les textes et erreurs en dessous. Ajoutez des étiquettes : `Résoudre avec MATLAB` et `Ouvrir le PDF` près des options si leurs champs vous semblent trop courts. Conservez les noms internes resoudre/afficherPDF du typedef.

Zone de dessin Image originale : environ 650×620 pixels et fond blanc. Clic droit `Visible Items` : activez les barres de défilement horizontale/verticale si disponibles. Cela affiche les exemples 6×6/8×8 à taille lisible sans redimensionner la matrice de pixels. Le PNG sélectionné est un **fichier**, pas un dossier. Le programme s'exécute une fois avec la flèche Run.

17.2 Dossier et validation de l'entrée

1. Hors de toute Case : N1 `Current VI's Path`→N2 `Strip Path` ; **stripped path** de N2→P11 et fil Dossier utilisé partout. N2.name est inutilisé. Le VI doit avoir été enregistré, sinon ce chemin est indéfini. Aucun chemin personnel constant n'entre dans le diagramme.
2. Garde S0 sélectionnée par P4. Dans Error : P7 picture vide ; P8/P9/P14 chaînes vides ; P12 I32 0 ; P13 tableau Boolean 2D vide ; P15 Path vide ; P10=P4.
3. Dans S0.No Error : N3 `Path To String(P1)`→N4 `TexteCheminValide.vi`, error in=P4. `Strip Path(P1)` donne name ; `String Length(name)` donne L. `String Subset(name,offset=L-4,length=4)` puis `To Lower Case` puis `Equal?` avec `.png`. AND de cette égalité, de `Greater?(L,4)` et de N4.Valide ; NOT→N6 ErreurSi(7010, `Choisir un fichier PNG sans caractere de controle.`,N4.error out). Pour L<4, String Subset peut renvoyer une chaîne partielle, mais Greater? impose le refus. Cela ne lit pas le contenu : LireImage refusera BadBinairo.png.
4. V1 LireImage : PNG=P1,error in=erreur précédente. V1.Image→tunnel Image ; Bitmap→P13 via tunnel et V2 ComputeRowsCols.Bitmap ; V1.error out→V2.error in. V2.n→tunnel Taille. Garde S1 sur V2.error out.
5. S1.Error : MatriceTexte/Diagnostic/Script vides, PDF path vide,error out=V2.error out. Image/Bitmap/Taille restent raccordés depuis V1/V2, ce qui aide à diagnostiquer une géométrie invalide.

17.3 Déterminer les résultats et supprimer les anciens

Dans S1.No Error : N7 Strip Path(P1)→name ; N8 String Length(name) ; N9 Subtract(N8,4) ; N10 String Subset(name,offset 0,length=N9) ; N11 Concatenate Strings(N10, `.pdf`) ; N12 Build Path(base=Dossier,name=N11)→P15 via les tunnels. Cette expression correspond à CheminPdfBinairo.m. Un PNG nommé `a.png` donne `a.pdf`.

N13 Build Path(Dossier, `solve.m`). V3 SupprimerResultat sur N13 avec error in=V2.error out ; V4 SupprimerResultat sur N12 avec error in=V3.error out. Cela protège contre les résultats d'une exécution précédente. Ne supprimez ni PNG ni source MATLAB. CellValue est supprimé par LireCase avant chaque appel OCR.

17.4 Boucle de lignes puis boucle de colonnes

Dans S1.No Error, posez F1 For Loop extérieur ; V2.n→F1.N. **Aucun auto-indexing**, aucun parallélisme, aucun arrêt conditionnel. Ajoutez trois Shift Registers :

- `matrice` String, initialisé avec chaîne vide ;
- `diagnostic` String, initialisé avec chaîne vide ;
- `erreur` cluster, initialisé avec V4.error out.

Faites entrer par tunnels non indexés Bitmap, Lignes, Dossier, Seuils, n. À l'intérieur F1, posez F2 For Loop ; n→F2.N. Trois Shift Registers : `ligneTexte` chaîne vide, `diagnostic` initialisé avec F1.diagnostic gauche,

`erreur` initialisé avec F1.erreur gauche. Faites entrer Bitmap,Lignes,Dossier,Seuils et **F1.i comme indiceLigne** par tunnels non indexés. F2.i est l'indiceColonne.

À l'intérieur F2 : Case C1 sélectionnée par son registre erreur gauche.

C1.Error : ligneTexte gauche→tunnel ligneTexte ; diagnostic gauche→tunnel diagnostic ; erreur gauche→tunnel erreur. Pas de sous-VI d'OCR dans cette case.

C1.No Error :

1. B1 Bundle By Name, cluster de base créé depuis LireCase.Indices, ordre ligne puis colonne ; F1.i transmis→ligne ; F2.i→colonne.
2. V5 LireCase : Bitmap/Lignes/Dossier/Seuils depuis leurs tunnels ; B1→Indices ; F2.erreur gauche→error in.
3. N14 Equal?(F2.i,0) ; N15 Select s=N14,t=chaîne vide,f=chaîne contenant **un espace**.
4. N16 Concatenate Strings trois entrées : F2.ligneTexte gauche, N15, V5.ValeurMatlab. Résultat = proposition de nouvelle ligne.
5. N17 Concatenate Strings trois entrées : F2.diagnostic gauche, V5.Diagnostic, constante LF (`\n` en Backslash Codes Display).
6. Case C2 sélectionnée par V5.error out. C2.No Error→N16 vers sortie ligneTexte. C2.Error→F2.ligneTexte gauche vers sortie ligneTexte (ne pas ajouter de fausse valeur). Dans **les deux** cases C2 : N17→diagnostic, V5.error out→erreur.
7. C2 sorties→C1 sorties ; C1 ligneTexte/diagnostic/erreur→les trois registres droits de F2. Tous les registres sont alimentés à chaque tour.

Après F2 mais encore dans F1 : N18 Concatenate Strings trois entrées : F1.matrice gauche, **valeur finale de F2.ligneTexte**, constante `;\n` saisie en Backslash Codes Display. N18→F1.matrice droite ; F2.diagnostic final→F1.diagnostic droite ; F2.erreur final→F1.erreur droite.

Une erreur peut laisser une matrice partielle affichée pour diagnostic, mais elle empêche GenererScript/LancerMatlab grâce aux gardes. Les tours restants des boucles ne font plus d'accès disque ou de processus. Les boucles n'ont pas besoin d'un arrêt anticipé pour être sûres.

17.5 Générer lancer vérifier

Après F1, toujours dans S1.No Error :

Sous-VI ou nœud	Câblage
V6 GenererScript	MatriceTexte=F1.matrice final ; PNG=P1 ; Options=P3 ; Dossier=N2.stripped path ; error in=F1.erreur final
V7 LancerMatlab	Dossier=N2.stripped path ; ExecutableMatlab=P5 ; LanceurFourni=P6 ; error in=V6.error out
V8 VerifierPDF	CheminPDF=N12.path ; error in=V7.error out

N19 Concatenate Strings	F1.diagnostic final puis V7.Diagnostic

F1.matrice final→Matrice reconnue ; V6.Script→Script genere ; N19→Diagnostic ; V8.error out→P10, en passant par les tunnels S1 puis S0. P7/P13/P12 reçoivent les valeurs Image/Bitmap/n de l'étape 17.2 via S0. P15 reçoit N12.path via S1/S0. **Un chemin affiché dans PDF attendu n'est pas un indicateur de succès** : seul error out clair après VerifierPDF autorise à considérer le nouveau fichier produit.

Pour rendre la panne immédiatement visible, ajoutez une LED `Erreur` non connectée au connector pane : `Unbundle By Name status` sur le fil final error out→LED. Ajoutez près du cluster : `Si Erreur est allumée, ne pas utiliser les anciens fichiers.` Ne masquez pas le code ou la source du cluster erreur.

Enregistrez. La flèche doit être entière. Sélectionnez le PNG6×6 ; laissez Executable MATLAB vide si `where matlab` trouve MATLAB, sinon choisissez matlab.exe *(modifié le 30.10)* ; choisissez le mode direct (Lanceur fourni=FALSE). Seuils88/90/90, deux options TRUE. Lancez **une fois**. Attendez la fin avant de recliquer. La reconnaissance remplace Cell.bin à chaque cellule : seule la dernière reste disponible à la fin.

18 Lire les résultats attendus

18.1 Exemple 6 par 6

Attendez Taille grille=6 et la matrice suivante dans le script (espaces et fins de lignes peuvent différer, pas les valeurs) :

```
% Projet Binairo - ME-213
% Auteurs : Louis Pedelaborde, Romeo Mugnier de Almeida, Raphael Pical
B = [
NaN 0 NaN NaN NaN NaN;
NaN NaN 0 NaN 0 NaN;
NaN NaN NaN NaN 1 0;
1 1 NaN NaN NaN NaN;
NaN 0 NaN 0 NaN NaN;
NaN NaN NaN NaN 0 NaN;
];
fichierSource = 'Binairo_6x6.png';
disp(B);
if true
    [S, ok] = SolveBinairo(B);
else
    S = B;
    ok = GrilleBinairoValide(B);
end
DisplayBinairo(B, S, fichierSource, ok);
if true
    uiopen(CheminPdfBinairo(fichierSource), 1);
end
if ok
    fprintf('Binairo traite.\n');
else
    fprintf('Grille contradictoire ou sans solution.\n');
end
```

Le PDF doit contenir tous les indices initiaux en noir gras et toutes les valeurs ajoutées en bleu gras. La date/heure doit correspondre à cette exécution. Fermez le lecteur PDF avant un nouvel essai si Windows verrouille le fichier : une erreur d'accès ne doit jamais être ignorée.

Variante rectangulaire : `Binairo_6x6_rect.v2.png` contient également 6 lignes et 6 colonnes, avec les mêmes indices que la matrice ci-dessus. L'image mesure 545 pixels de large et 360 de haut : les cellules sont rectangulaires. Traitez le PNG original sans le redimensionner. Le script généré doit contenir `fichierSource = 'Binairo_6x6_rect.v2.png';` et produire **`Binairo_6x6_rect.v2.pdf`**, en conservant le point dans `.v2`. Comparez toutes les cases reconnues avant de juger la solution. Effectuez V26 sur la VM et sauvegardez les quatre fichiers selon §21.2 ; ce résultat natif reste à mesurer.

18.1b Exemple 8 par 8 et grille impossible

Avant d'observer une solution, comparez **toutes les cases reconnues** à ces matrices. Ne les mettez jamais dans GenererScript à la place du résultat OCR.

8×8, 15 indices :

```
B = [NaN 0 0 NaN NaN NaN NaN 0;
NaN NaN NaN NaN NaN NaN NaN NaN;
1 NaN 1 NaN 0 NaN NaN NaN;
NaN NaN NaN NaN NaN NaN NaN 0;
1 NaN NaN NaN 1 NaN NaN NaN;
NaN NaN NaN NaN 1 NaN 1 NaN;
NaN NaN 0 NaN NaN NaN NaN NaN;
NaN 0 NaN NaN NaN NaN 0 0];
```

4×4 Bad, quatre indices :

```
B = [NaN 0 0 NaN;
NaN NaN NaN NaN;
NaN 0 0 NaN;
NaN NaN NaN NaN];
```

Le 4×4 a une géométrie correcte ; après déductions, il n'a aucune solution. Avec Résoudre=TRUE, attendez `ok=false` et un PDF portant `== Error ==`. Un PDF de grille résolue indiquerait une perte d'indices ou une erreur.

18.2 Quatre combinaisons des options

Résoudre	Ouvrir PDF	Résultat attendu
TRUE	TRUE	PDF de la solution produit et ouvert
TRUE	FALSE	PDF de la solution produit, pas ouvert automatiquement
FALSE	TRUE	PDF de la grille initiale produit et ouvert, cases vides conservées
FALSE	FALSE	PDF de la grille initiale produit, pas ouvert automatiquement

Dans les deux dernières lignes, aucune existence de solution n'est certifiée. Pour une grille initiale déjà contradictoire, le PDF doit afficher `== Error ==`. L'option Résoudre FALSE ne veut pas dire « ne pas lancer MATLAB » : MATLAB reste responsable du dessin.

18.3 Tester l'ouverture PDF avec le lancement réel

`uiopen(chemin,1)` est proposé dans l'addendum actuel (`P00.PPI_Projet_soumission.26.2.pdf`, page physique 14). Son ouverture du lecteur PDF en mode `-batch` doit néanmoins être testée sur la VM : elle n'a pas été exécutée ici.

1. Fermez tout PDF ouvert. Dans GenererScript.vi, générez le script du checkpoint 2×2 avec Résoudre=TRUE, Ouvrir PDF=FALSE et le nom `Grille.png`.
2. Lancez-le par LancerMatlab.vi en mode direct, avec Executable MATLAB vide, puis une fois avec le vrai matlab.exe *(modifié le 30.10)*. Attendez la fin. Vérifiez un code de retour 0 et l'existence de Grille.pdf. Aucun lecteur ne doit s'ouvrir automatiquement.
3. Régénérez le même script avec Ouvrir PDF=TRUE. Relancez par le même VI. Vérifiez le code 0 **et l'ouverture visible du PDF** ; un fichier créé sans lecteur ouvert ne valide pas l'option.
4. Répétez les deux essais avec Résoudre=FALSE, puis depuis un dossier contenant des espaces. Utilisez aussi le nom `L'exemple_test.png` pour vérifier les apostrophes et le caractère souligné.
5. Notez la version MATLAB, le code de retour, la sortie d'erreur et l'observation du lecteur. Si le PDF existe mais ne s'ouvre pas, vérifiez d'abord son ouverture par double-clic dans Windows. Ne marquez pas V05 réussi tant que les quatre combinaisons ne fonctionnent pas par le lancement `-batch` réel.
6. **Solution de repli si le PDF existe mais ne s'ouvre pas en `-batch`** *(modifié le 30.10)* : l'addendum (p. 14) prévoit aussi l'ouverture depuis LabVIEW par System Exec. Dans GenererScript.vi, mettez la chaîne `false` sur l'entrée t de N5, pour que solve.m n'appelle plus uiopen. Dans BinairoSolver.vi, après V8 VerifierPDF, posez une Case sélectionnée par AND(Options.afficherPDF, NOT(status de V8.error out)). Dans TRUE : `Path To String(N12.path)` → `Format Into String` avec le format `cmd /c start "" "%s"` →System Exec (working directory=Dossier, wait until completion?=FALSE, error in=V8.error out) ; son error out devient l'erreur finale. Dans FALSE : V8.error out traverse. Refaites alors les étapes 3 et 4.

18.4 Deux types d'échec à ne pas confondre

- `Binairo_4x4_Bad.png` peut avoir une géométrie correcte tout en étant contradictoire : OCR complet, MATLAB terminé, PDF `== Error ==` attendu.
- `Binaro_5x6_Bad.png` a un nombre de lignes/colonnes incompatible : erreur de géométrie **avant** l'OCR et avant le lancement MATLAB.

19 Batterie finale de validation native

Pour chaque ligne : notez date, version des logiciels, PASS/FAIL et observation réelle dans `Preuves/validation.txt`. Ne cochez pas une ligne parce qu'elle est écrite dans la recette. Restaurez les noms/options après chaque essai.

Essai	Action exacte	Réussite attendue
V01	PNG6×6,88/90/90,TRUE/TRUE	matrice du chapitre 18, PDF complet, erreur claire
V02	PNG8×8, mêmes options	n=8, tous les indices conservés, règles de la solution vérifiées
V03	PNG4×4_Bad	n=4, PDF == Error == ; aucun faux résultat résolu
V04	PNG5×6_Bad	erreur géométrie, pas de lancement MATLAB
V05	Les quatre lignes du tableau 18.2, par le lancement - batch réel ; suivre 18.3	code 0, PDF correct, ouverture visible seulement si demandée
V06	Sélectionner un chemin PNG inexistant	erreur de lecture visible
V07	Sélectionner **BadBinairo.png fourni sur Moodle** ; puis un texte abc renommé faux.png	erreur PNG visible, aucun lancement
V08	Champ PNG vide puis fichier `.jpg`	erreur avant traitement
V09	Renommer **votre copie** OCR.exe en OCR_absent.exe ; lancer 6×6	erreur de lancement OCR, aucun ancien CellValue exploité ; remettre OCR.exe
V10	Dans chaque champ, essayer0 puis 100	erreur LabVIEW 7004 avant OCR ; en appel C direct, code 2
V11	Dans chaque champ, essayer88.5,-1,101,NaN,Inf ; puis 1 et 99	invalides refusés ; 1/99 admis par CommandeOCR ; remettre 88/90/90
V12	Fermer le programme ; créer un dossier nommé CellValue.txt dans un dossier Execution de test	erreur sans suppression du dossier ; l'enlever manuellement après l'essai
V13	Créer un dossier nommé solve.m dans ce même dossier d'essai	erreur avant MATLAB, dossier conservé
V14	Créer un dossier nommé Binairo_6x6.pdf dans le dossier d'essai	erreur avant lancement, dossier conservé

V15	Mode direct : sélectionner un matlab.exe inexistant ; puis, sur un compte où `where matlab` ne trouve rien, laisser le champ vide *(modifié le 30.10)*	erreur lancement visible et pas de succès PDF
V16	Essai isolé de LancerMatlab avec solve.m contenant `error('Essai volontaire');`	erreur script réelle détectée ; régénérer ensuite le script normal
V17	Essai isolé : supprimer le PDF, faire lancer un script qui ne produit aucun PDF, puis VerifierPDF	erreur 7007 ou erreur native fichier absent
V18	Relancer V01 juste après un échec	succès nouveau, pas de reprise de fichier périmé
V19	Fermer LabVIEW, copier Execution dans un autre dossier avec espaces, rouvrir BinairoSolver.vi et sélectionner PNG	succès identique, pas de chemin absolu figé
V20	Dans ce dossier, rendre un fichier résultat non inscriptible avec les permissions du compte de test, puis lancer	erreur écriture transmise ; restaurer les permissions
V21	Tous les checkpoints EtendueNoire/IntervallesNoirs/rectangle et binaire asymétrique	indices et octets exacts
V22	Mode direct : succès et erreur volontaire, avec attente effective ; répéter pour le mode fourni seulement s'il est installé	attend la fin et transmet l'erreur ; captures conservées
V23	C : BadCell_1, un pixel manquant, un pixel supplémentaire, cellule non vide sans chiffre ; consulter les choix de warning du chapitre 3	comportements et messages conformes aux choix documentés
V24	Géométrie/binaire : dimensions 49,50,1000,1001 sur chaque axe ; suivre G4.4	49/1001 refusées ; 50/1000 acceptées ; pas d'inversion largeur/hauteur
V25	Exporter chaque jeu de résultats avant le suivant ; assembler et réextraire l'archive selon 21	fichiers présents et programme portable
V26	`Binairo_6x6_rect.v2.png` original, 88/90/90, TRUE/FALSE, puis TRUE/TRUE	n=6, matrice de §18.1, cellules rectangulaires sans inversion hauteur/largeur ; PDF `Binairo_6x6_rect.v2.pdf` correct, ouverture conforme à l'option ; quatre résultats sauvegardés selon §21.2

Pour V20, l'attribut « lecture seule » d'un dossier Windows ne prouve pas une interdiction d'écrire : utilisez un fichier protégé ou les permissions réelles, sans modifier les permissions de l'installation de cours. Si votre compte ne permet pas ce test, marquez-le non exécuté et utilisez V12–V14 pour vérifier déjà les échecs d'ouverture.

Pour V02, comparez l'original à la matrice reconnue avant de juger la solution. Pour chaque ligne et colonne de la solution : moitié 0/moitié 1, jamais trois identiques consécutifs, aucune ligne complète dupliquée, aucune colonne complète dupliquée. La photo de l'image d'origine et le PDF final doivent permettre de vérifier les couleurs des indices.

19.1 Ajouter des tests de géométrie sans écrire de code externe

Dans un VI de test séparé : `Initialize Array` de Boolean FALSE de taille 46×46, puis `Replace Array Subset` pour dessiner des traits complets à lignes/colonnes 0,15,30,45 avec TRUE (utilisez des vecteurs TRUE produits par Initialize Array de longueur 46). Le tableau représente 3×3 cases et doit être refusé pour nombre impair. Répétez avec dimensions 31×46 et traits de lignes 0,15,30, colonnes 0,15,30,45 : 2×3 doit être refusé. Une matrice totalement FALSE doit être refusée comme absence de grille. Les instructions géométrie donnent aussi des tests unitaires plus simples de transitions.

Si vous créez des images complémentaires avec un logiciel de dessin, gardez-les comme tests, sans les confondre avec les fichiers du professeur. Une grille photographiée en perspective n'est pas un cas implicitement pris en charge : l'algorithme demandé suppose des traits alignés avec les axes.

20 Si quelque chose ne marche pas

Ne modifiez pas dix fils à la fois. Arrêtez-vous au **premier checkpoint faux**.

Symptôme	Vérification dans cet ordre
Flèche cassée	Error List ; borne non câblée ; type exact ; sortie Case oubliée ; tunnel indexé par erreur
Bitmap inversé	chapitre 7.4 et les deux pixels de calibration ; ne pas inverser dans plusieurs VIs
n vaut 0,1 ou nombre incohérent	Bitmap correct ? marges prises en compte ? balayage top+10/left+10 ? IntervallesNoirs inclut-il les bords ?
Cell.bin refusé	lire les huit premiers U8 ; hauteur puis largeur *(modifié le 30.10)* ; byte order2 ; deux prependFALSE ; pixels U8 et non bits compactés
OCR code 5 partout	ordre hauteur puis largeur de l'en-tête, §9.1 *(modifié le 30.10)* ; polarité ; dimensions ; découpage G4 à l'intérieur des traits ; seuils 88/90/90 ; police fournie présente
Fichier OCR jamais trouvé	working directory de System Exec ; OCR.exe réellement Windows ; nom exact Cell.bin ; file close avant appel
Tous les indices deviennent NaN	vérifier 88/90/90, la polarité (7.4) et G4 : le chiffre 32×32 doit tenir entier dans la case *(modifié le 30.10)* ; comparer les matrices 18.1/18.1b
Matrice transposée	F1.i=ligne, F2.i=colonne ; Array Subset row puis col ; ne pas transposer le bitmap de production
MATLAB rejette les options	le script doit contenir `true` et `false`, pas 0/1 ni"TRUE"
MATLAB fonction introuvable	sept fichiers `.m` dans Execution ; working directory du lanceur ; nom/casse du fichier
MATLAB ne démarre pas, champ vide	`where matlab` dans `cmd` ; sinon choisir matlab.exe ou faire ajouter MATLAB au PATH, puis relancer LabVIEW *(modifié le 30.10)*
PDF ancien malgré panne	supprimer l'ancien avant lancement ; attendre la fin ; vérifier error out, return code et fichier nouveau
PDF verrouillé	fermer le lecteur de PDF avant de réessayer ; ne pas ignorer l'erreur de suppression
Le lanceur fourni a d'autres bornes	capture Front Panel, connector pane, Context Help et diagramme ; ne pas adopter les bornes du Billard

20.1 Ce qu'il faut renvoyer pour une correction précise

1. Enregistrez tous les VIs. Ne relancez pas une autre image avant d'avoir copié les résultats : cela écraserait Cell.bin/CellValue.txt.
2. Copiez dans un dossier Preuves daté : VIs et `.ctl` actuels, PNG exact utilisé si partage permis, Cell.bin, CellValue.txt si présent, solve.m si présent et PDF si présent.
3. Ajoutez une capture de la face avant principale et du diagramme du premier sous-VI défaillant. Les identifiants d'objets doivent rester lisibles.
4. Copiez intégralement error out.status/code/source et Diagnostic dans un texte. Une seule capture « erreur » sans code ni source ne suffit pas.
5. Pour un fil impossible, ajoutez Context Help du nœud et de la borne visée. Pour une panne de lancement, ajoutez le texte exact de la commande, return code, stdout, stderr.
6. Fermez LabVIEW puis archivez ce dossier de preuves. Le dossier ne constitue pas encore l'archive de remise.

21 Assembler le rendu, étape par étape

L'addendum du 7 octobre fixe ce contenu. La recette, un binaire Mac ou les seuls fichiers sources ne constituent pas un rendu Windows validé.

21.1 Fermer la validation native

1. Exécutez chaque essai du chapitre 19, y compris V26, sur la VM avec l'`OCR.exe` optimisé compilé au §1.1. Après toute recompilation, refaites ces essais avant de remplacer l'exécutable du rendu. Dans Preuves/validation.txt, inscrivez le résultat observé, la date et les versions. Un essai non exécuté reste indiqué comme tel.
2. Vérifiez les descriptions et auteurs dans les VIs. Rangez les fils ; laissez les erreurs visibles. Fermez les fichiers PDF ouverts.
3. Remettez les seuils **88/90/90**, Options TRUE/TRUE, Lanceur fourni FALSE et une erreur claire. `Edit > Make Current Values Default`, puis `Ctrl+S`, pour les typedefs et le principal. Les chemins se choisissent sur la face avant, sans constante personnelle dans le diagramme.
4. Fermez LabVIEW et copiez les VIs, typedefs et projet dans votre sauvegarde persistante. Le principal se nomme exactement **BinairoSolver.vi**.

21.2 Sauver les résultats avant qu'ils soient écrasés

1. Créez `Preuves/6x6`, `Preuves/6x6_rect_v2`, `Preuves/8x8`, `Preuves/4x4_Bad`, `Preuves/5x6_Bad`, `Preuves/BadBinairo`.
2. Exécutez 6×6 avec 88/90/90, TRUE/FALSE. Vérifiez la matrice et le PDF. **Avant toute nouvelle exécution**, copiez de Execution vers Preuves/6x6 : `Cell.bin`, `CellValue.txt`, `solve.m`, `Binairo_6x6.pdf`, plus le diagnostic observé.
3. Exécutez **`Binairo_6x6_rect.v2.png`** avec 88/90/90, TRUE/FALSE. Vérifiez la matrice de §18.1 et le PDF. **Avant toute nouvelle exécution**, copiez de Execution vers `Preuves/6x6_rect_v2` : **`Cell.bin`**, **`CellValue.txt`**, **`solve.m`** et **`Binairo_6x6_rect.v2.pdf`**, plus le diagnostic observé. Ce jeu est distinct de celui du 6×6 original.
4. Répétez pour 8×8 puis 4×4_Bad, chacun dans son sous-dossier. Le dernier PDF doit contenir `== Error ==`. Les fichiers Cell.bin/CellValue correspondent à la dernière cellule de la grille ; ne les décrivez pas comme toutes les cellules.
5. Pour 5×6 etBadBinairo, conservez la capture et le diagnostic de refus. Aucun solve.m/PDF réussi n'est attendu : **ne copiez pas un ancien résultat comme preuve de ces cas**.
6. Conservez les basenames requis dans chaque sous-dossier. Les PNG sources restent dans Images et ne seront pas ajoutés à l'archive de remise.

21.3 Remplir la documentation courte et réunir l'archive

1. Après les essais dans la VM, rédigez la brève documentation selon le modèle Moodle `Ex. Documentation projet.pdf`, puis exportez-la en PDF. Indiquez les auteurs, versions VM/Windows/compilateur/MATLAB/LabVIEW, rôle des fichiers, chaîne PNG → LabVIEW → C → MATLAB → PDF, algorithmes, formats, seuils et politiques de warning du chapitre 3, résultats réellement

observés et limites restantes. Mentionnez les codes C 0,2,3,4,5,6,7, l'emplacement de CellValue.txt (dossier de Cell.bin, vidé après une erreur), la lecture d'une case vide comme `1` si les seuils des chiffres sont très bas (les chiffres sont testés d'abord, Ed #28), le sens de l'option Résoudre=FALSE (MATLAB dessine la grille initiale sans la résoudre), le lancement de MATLAB par le PATH et le comportement du contrôleur après une erreur. *(modifié le 30.10)* Ne remplacez pas « non exécuté » par « réussi » sans essai. Aucun rapport final prérempli n'est fourni dans le dépôt.

2. Exportez **les conversations complètes d'aide IA en PDF**, y compris l'audit et les corrections. Le rapport d'audit ou un résumé ne remplace pas le transcript. Déclarez aussi toute autre source de code et toute coopération significative avec un autre groupe, si applicable ; n'inventez pas une déclaration d'absence à leur place.
3. Créez un dossier de remise portant les noms **Pedelaborde_MugnierDeAlmeida_Pical**, avec les trois sous-dossiers **c**, **matlab** et **temp**. Dans c, conservez les cinq fichiers C/header et les fichiers projet créés avec votre IDE dans la VM ; dans matlab, les sept fonctions `.m`. Dans `temp/Execution`, copiez le contenu utile du dossier Execution testé : principal, tous ses sous-VIs, quatre typedefs, projet LabVIEW, sept fonctions MATLAB, cinq fichiers C/header, **OCR.exe Windows compilé et testé**, ainsi que les fichiers projet associés réellement créés dans la VM. Les copies d'exécution doivent correspondre aux dernières sources de c/matlab. Conservez leurs chemins relatifs ; aucun projet ne doit dépendre du dossier d'un développeur.
4. Dans temp, gardez la recette, créez `docs` pour la documentation courte PDF et les références nécessaires, puis ajoutez `Transcripts` avec les conversations complètes en PDF et `Preuves` avec les résultats de chaque séquence. Le dossier temp regroupe provisoirement le reste du projet : ne le supprimez pas avant remise, car il contient les VIs et les pièces exigées. **Excluez Images et les PNG de test**, les dossiers `.git`, `.vs`, les fichiers de compilation intermédiaires et les exécutables pour une autre plateforme. Les exemples binaires d'entrée du professeur servent aux tests locaux ; les résultats générés sont dans Preuves.
5. Comparez le contenu avec cette liste, puis compressez le dossier en `Pedelaborde_MugnierDeAlmeida_Pical.zip`. Conservez une copie datée avant toute soumission.

21.4 Vérifier exactement ce qui sera évalué

1. Extrayez le ZIP dans **un autre dossier avec espaces**, idéalement sur une autre machine/VM prévue par le cours.
2. Ouvrez `temp/Execution/BinairoSolver.vi` depuis le dossier extrait, sélectionnez le PNG conservé à part et le MATLAB de cette machine, puis utilisez la flèche Run. Vérifiez 6×6, `Binairo_6x6_rect.v2.png`, 8×8, les cas invalides et l'absence de dépendance manquante.
3. Après la soumission Moodle, **retéléchargez l'archive déposée**, extrayez-la ailleurs et répétez le contrôle. Seule la dernière soumission est retenue. Ce document ne dépose rien à votre place.
4. Relisez toute nouvelle annonce Moodle avant la remise. L'addendum du 7 octobre est intégré ; une nouvelle instruction postérieure devra être comparée explicitement.

22 Références à consulter en cas de différence

Les numéros suivants sont les **pages physiques du PDF**, pas nécessairement les nombres imprimés sur les diapositives.

Source du cours	Pages	Exigence utilisée
P00.PPI_Projet.2026.2.pdf	6,15–18,23–27,31–36	noms, format binaire/texte, interface, orchestration, résolution, PDF, erreurs
Exo.Proj.Binairo.LabVIEW.1.pdf	3–10,13–16	lecture/conversion image, indexation, co-localisation, System Exec, géométrie
Exo.Proj.Binairo.LabVIEW.2.pdf	2–7	matrice générée, script, lanceur fourni, erreurs
Exo.Proj.Binairo.Matlab.1.pdf	1–2	résultat PDF et fonctions d'affichage
P0.PPI_Projet_soumission.26.1.pdf	3–12,14–16	rendu, erreurs, dimensions 50..1000, seuils entiers 1..99 et 88/90/90, noms, defaults, PDF
LV1 et LV2, versions 2026	annexe H	structures, tableaux, chaînes, fichiers, erreurs et sous-VIs

Les exercices de ce tableau sont des aides secondaires ; leurs noms de sous-VIs et leur lanceur ne deviennent pas des obligations absentes de la donnée.

Documentation primaire des fonctions NI utilisées, pour vérifier leurs bornes avec Context Help dans votre version :

- Write to Binary File : refnum partagé, octets et taille préfixée.
- Unflatten Pixmap : sortie selon profondeur et indices de palette.
- Picture to Pixmap : profondeur, rectangle et fond.
- Lancer directement un exécutable : chemin complet de l'exécutable avec System Exec.
- System Exec et commandes : lancement et attente.
- Codes de localisation : forcer le point avec `%.;`.
- Lancement MATLAB sous Windows : options `-wait` et `-batch`.
- Scan From String et Format Into String : types du scan, reformatage et échappement des barres inverses.
- Check if File or Folder Exists : propager error out.
- Color to RGB : composantes U8 à convertir avant addition.
- File/Directory Info et Delete : contrôle des fichiers et suppression non récursive.

23 Contrôle de couverture avant de déclarer la partie terminée

L'addendum du 7 octobre prime pour les points qu'il fixe ; la donnée 2026.2 et les exercices compatibles complètent les exigences. Les pages sources sont au chapitre 22. Ce tableau relie les exigences à leur construction et à leur vérification.

Exigence	Où la construire	Preuve à obtenir
VI principal BinairoSolver.vi et interface utilisable	17.1	source PNG, deux options et erreur visibles ; V01/V05
Lecture PNG et conversion native couleur→noir/blanc	7	dimensions, polarité et image affichée ; checkpoint 7.5/V07
Nombre pair et égal de lignes/colonnes ; traits aux bords	annexeG	V04/V21, grille synthétique 2×2, refus 3×3 et 2×4
Comptage et découpage de la donnée, noms de sous-VIs repris de l'exercice	annexeG	résultats nommés et coordonnées mesurées ; V21
Découpage avec bord mesuré ; format binaire correct	9/12/G4	asymétrie 55×60, octets attendus, cellule 72×68 pour la première case 6×6 *(modifié le 30.10)*
Appel C paramétré, séquentiel, résultat réellement lu	10–12/17.4	V01,V09,V10 ; matrice comparée à l'image
Script MATLAB produit depuis toute la grille reconnue	13/17	solve.m réellement généré, valeurs 0/1/NaN exactes
Lancement MATLAB et détection des erreurs (donnée p. 26,36)	15.2 ; option 15.3 seulement si retenue	attente effective, succès et panne volontaire ; V22
Résolution et affichage commandés par les deux options	13/18.2	les quatre combinaisons V05
PDF au nom du PNG, style et résultat conformes	16/18	contrôle visuel des indices/couleurs/titre/date, V01–V03
Pas de PNG, OCR absent/en erreur, écriture script impossible, MATLAB absent/script en erreur	5/6/12/15/17	V06–V17 : erreur affichée, aucune étape suivante exécutée
Pas de reprise de résultats périmés	6/12/16/17	V09,V12–V18, nouveau PDF après succès
Chemins portables et fichiers co-localisés	1/17.2	V19 dans un autre dossier avec espaces

Sous-VIs, connecteurs, types et valeurs par défaut cohérents	2/3, chaque interface	absence de flèche cassée ; ouvrir chaque sous-VI et contrôler ses défauts
Documentation et auteurs ; diagrammes lisibles	2.6/21	aide de chaque VI, contrôles décrits, fils rangés
Exécution effective dans votre environnement	19	résultats natifs enregistrés pour tous les tests applicables
Derniers compléments de consignes/erreurs/connecteur	21	comparaison avec Moodle avant remise, adaptation si un contrat nouveau est fourni

État de cette recette : le C du 30 octobre est compilé localement. Avec le découpage à l'intérieur des traits, les 116 cellules des trois images carrées sont toutes reconnues correctement à 88/90/90 pour quatre conversions RGB d'essai (464 appels), et Binaro_5x6_Bad est refusé (6103). *(modifié le 30.10)* Cette mesure indépendante ne remplace pas Picture to Pixmap ni les VIs Windows. Les matrices attendues figurent au chapitre 18. Les tests natifs restent ceux du chapitre 19.

La partie LabVIEW peut être déclarée terminée lorsque les tests natifs sont réussis, le mode direct de lancement MATLAB fonctionne réellement, les fichiers nécessaires sont sauvegardés, et les dernières consignes officielles sont intégrées. L'inspection facultative du chapitre 14 n'est pas une condition de conformité à la donnée.

23.1 Ambiguïtés résiduelles

L'addendum fournit maintenant la liste des erreurs et la remise : ne demandez plus si ces documents existent. Deux points ne sont pas explicitement tranchés dans l'export : score brut contre marge lorsque les seuils diffèrent, et éventuel connecteur officiel du principal. Le code suit la procédure détaillée p. 18 (marge), les deux seuils par défaut étant identiques ; le connecteur de la recette est un choix documenté. Si le professeur donne une précision ultérieure, répercutez-la dans le code, la recette et les tests. Aucun message n'est envoyé par cette recette.

Annexe G — Détecter la grille et découper ses cases

Ce chapitre construit `EtendueNoire.vi`, `IntervallesNoirs.vi`, `ComputeCellRect.vi`, puis `ComputeRowsCols.vi`, dans cet ordre. Les deux derniers noms sont repris de l'exercice comme conventions compatibles. La donnée impose les opérations correspondantes (p. 25–27), sans fixer les noms de ces deux sous-VIs. Les deux premiers sont nos sous-VIs auxiliaires. `ErreurSi.vi` doit déjà exister : entrées `Condition` Boolean, `Code` I32, `Message` String, `error in` ; sortie `error out`. Il conserve une erreur entrante ; sinon il crée l'erreur demandée uniquement lorsque `Condition` est vraie.

Les opérations ci-dessous se font dans LabVIEW. Elles ne sont pas des tests déjà exécutés. Les résultats indiqués sont les résultats à obtenir avant de poursuivre.

Références : `Exo.Proj.Binairo.LabVIEW.1.pdf`, pages physiques 5–8 et 15–16 ; `P00.PPI_Projet.2026.2.pdf`, pages physiques 25–27. Le professeur demande de compter les traits, traiter ceux qui touchent le bord, vérifier un nombre pair et égal de lignes/colonnes, puis tenir compte des bordures lors du découpage. Nous mesurons chaque trait : une bordure de 5 pixels est donc prise en compte par sa largeur réelle, sans soustraire arbitrairement 5 partout.

G0. Conventions à appliquer à chaque fil

1. Enregistrer les nouveaux fichiers dans le même dossier de travail LabVIEW que le VI principal.
2. `Bitmap` est le tableau **Boolean 2D déjà normalisé** fourni par `LireImage.vi` : TRUE = noir, FALSE = blanc. Premier indice = ligne, deuxième indice = colonne. Toutes les coordonnées commencent à **0**.
3. Tous les nombres de ce chapitre sont des **I32**, y compris les constantes. Pour chaque constante numérique : clic droit → Representation → I32.
4. Les intervalles sont `[debut, fin)` : `debut` appartient au trait ; `fin` est le premier pixel après le trait. De même, `droite` et `bas` ne font pas partie du rectangle extrait. Largeur = droite−gauche ; hauteur = bas−haut. **Ne pas ajouter 1.**
5. Pour placer une primitive, Ctrl+Space, saisir son nom anglais imprimé ici, sélectionner le résultat exact, Entrée. Ctrl+H ouvre l'aide contextuelle. Les noms de ports priment sur leur position dans l'icône.
6. Dans les tableaux, `A → B, C` signifie deux branches issues du même fil. `.x/.y` sont les deux entrées d'une opération ; `.sortie` son unique résultat. `Index Array.tableau/index/element` désigne les ports natifs correspondants. Pour une matrice, `index0` est la ligne et `index1` la colonne.
7. Un calcul nommé dans un tableau est **un objet à placer**. Un test nommé est **une primitive de comparaison distincte**. Les constantes peuvent être branchées vers plusieurs objets. Les noms de repères servent d'étiquettes libres dans le diagramme, pas de noms à rechercher dans Quick Drop.
8. Pour chaque `Case Structure`, connecter le **cluster d'erreur complet** au sélecteur. Les cas s'appellent alors `Error` et `No Error`. Ne pas sélectionner `Use Default If Unwired`. Tous les tunnels de sortie doivent être câblés dans les deux cas.
9. Pour acheminer une valeur extérieure jusqu'à un objet situé dans un cas imbriqué, tirer directement le fil à travers les bords des structures : LabVIEW crée les tunnels d'entrée nécessaires. Ne pas utiliser de variables locales. Une valeur qui ressort d'un cas doit, elle, être câblée dans **tous** les cas.
10. Chaque boucle `For Loop` a le parallélisme désactivé, aucun terminal d'arrêt et aucun registre à décalage sauf lorsque ce chapitre en demande un. Activer l'indexation uniquement aux tunnels explicitement indiqués.
11. `Build Array` recevant plusieurs valeurs scalaires a `Concatenate Inputs` **désactivé**. Lorsqu'on concatène des tableaux, le texte le demande explicitement.
12. Aucun tableau d'indices géométriques n'est lu avant la validation de sa taille et de l'indice.
13. Les codes 6101–6104 sont des **codes de notre projet**, pas des numéros officiels du professeur ou de NI.
14. Pour chaque VI, compléter File → VI Properties → Documentation : rôle, sens des entrées/sorties, unités et erreurs. Ajouter les auteurs Louis Pédelaborde, Romeo Mugnier de Almeida, Raphaël Pical. Sauvegarder des entrées vides/0 et un cluster sans erreur avec Edit → Make Current Values Default, puis sauvegarder le VI.

G1. Vérifier les deux types et attribuer les connecteurs

Les deux fichiers .ctl ont déjà été créés au chapitre 3 : vérifiez-les avec les indications ci-dessous, **ne créez pas une seconde copie**. Leur dossier est Execution.

G1.1 `LignesGrille.ctl`

1. Dans une face avant vide, déposer un `Cluster`.
2. À l'intérieur, déposer quatre `Array` contenant chacun un contrôle numérique I32. Un seul indice affiché : tableaux **1D**.
3. Nommer exactement les quatre tableaux `debutX`, `finX`, `debutY`, `finY`.
4. Clic droit sur le bord du cluster → Reorder Controls in Cluster. Affecter respectivement les ordres 0,1,2,3.
5. Laisser les quatre tableaux vides : ne pas saisir de zéro dans une case.
6. Clic droit → Advanced → Customize. Dans l'éditeur de contrôle, choisir `Type Def.` ; enregistrer `LignesGrille.ctl` ; fermer l'éditeur en acceptant de remplacer le contrôle initial.

G1.2 `RectangleCellule.ctl`

Même procédure, mais cluster de quatre **scalaires I32** : `gauche`, `haut`, `droite`, `bas`, dans cet ordre. Valeurs par défaut 0. Enregistrer `RectangleCellule.ctl`. Il s'agit du rectangle **de notre programme**, pas du cluster rectangle natif des fonctions Picture ; ici l'extraction finale se fait avec `Array Subset`.

G1.3 Connecteurs des quatre sous-VIs

Pour chaque VI, afficher le connecteur en haut à droite de la face avant et choisir le motif de **12 bornes** : quatre à gauche, quatre à droite, deux en haut et deux en bas. Repérer L1–L4 de haut en bas à gauche, R1–R4 de haut en bas à droite, T1/T2 en haut, B1/B2 en bas. Cliquer la borne puis le contrôle/indicateur indiqué.

VI	L1	L2	L3	L4	R1	R2	R3	R4
`EtendueNoire.vi`	Bits	libre	libre	error in	Debut	Fin	libre	error out
`IntervallesNoirs.vi`	Bits	libre	libre	error in	Debuts	Fins	libre	error out
`ComputeCellRect.vi`	ligne	colonne	Lignes	error in	Rectangle	libre	libre	error out
`ComputeRowsCols.vi`	Bitmap	libre	libre	error in	n	Lignes	libre	error out

Toutes les bornes T1,T2,B1,B2 restent libres. Les bornes de données d'entrée utilisées sont Required ; `error in` Recommended. Ce motif est notre choix pour les **sous-VIs** ; il ne prétend pas reproduire un connecteur officiel du VI principal.

G2. `EtendueNoire.vi` — trouver le premier et le dernier pixel occupé

G2.1 Face avant

Créer et sauvegarder `EtendueNoire.vi`. Ajouter :

Nom	Sens	Type	Défaut
Bits	contrôle	Boolean 1D	vide
error in	contrôle	cluster d'erreur standard	aucune erreur
Debut	indicateur	I32	0
Fin	indicateur	I32	0
error out	indicateur	cluster d'erreur standard	aucune erreur

G2.2 Placer et câbler

1. Placer `S0`, une Case Structure commandée par `error in`. Créer trois sorties : `debut`, `fin`, `erreur` ; relier aux indicateurs correspondants.
2. Cas **Error** : constante I32 0 → `debut` et `fin` ; `error in` → `erreur`.
3. Dans **No Error**, placer les objets suivants :

Repère	Primitive	Câblage des entrées	Résultat
A	Array Size	Bits → array	longueur
P	Search 1D Array	Bits → 1D array ; TRUE → element ; 0 → start index	premier indice noir
R	Reverse 1D Array	Bits → array	tableau inversé
Q	Search 1D Array	R.sortie → array ; TRUE → element ; 0 → start index	dernier noir vu depuis la fin
F	Subtract	A.longueur → x ; Q.index → y	fin exclusive
T	Less?	P.index → x ; 0 → y	absence de noir

E	ErreurSi.vi	T.sortie → Condition ; 6102 → Code ; `Aucun pixel noir sur l'axe examine.` → Message ; error in → error in	erreur validée
S1	Case Structure	E.error out → sélecteur	validation finale

4. Dans `S1.No Error` : P.index → sortie `debut` ; F.sortie → sortie `fin` ; E.error out → sortie `erreur`.
5. Dans `S1.Error` : 0 → `debut` et `fin` ; E.error out → `erreur`.
6. Relier les trois sorties de S1 aux trois sorties de S0. Aucun résultat calculé en cas d'erreur ne contourne S1.
7. Il n'y a **aucune boucle** ni registre à décalage dans ce VI. Les sorties inutilisées de Search 1D Array restent non connectées.

G2.3 Vérifier avant de continuer

Bits	Debut	Fin	Erreur
F,F,T,T,F,T,F	2	6	aucune
T	0	1	aucune
F,F	0	0	6102
tableau vide	0	0	6102

Injecter également `error in = status TRUE, code 123, source "essai"` : retrouver exactement cette erreur, avec Debut=Fin=0. Ne pas remplacer l'erreur 123 par 6102. Sauvegarder.

G3. `IntervallesNoirs.vi` — relever tous les traits, même aux bords

G3.1 Face avant et structure

Créer le VI. Contrôles : `Bits` Boolean 1D vide et `error in`. Indicateurs : `Debuts` et `Fins`, deux tableaux I32 1D vides, et `error out`. Attribuer le connecteur G1.

Placer `S0`, Case sur `error in`, sorties `debuts`, `fins`, `erreur`. Dans Error : tableau I32 1D vide vers `debuts` et `fins`, erreur entrante vers `erreur`.

G3.2 Cas No Error : objets et tous les fils

1. Créer une constante **tableau Boolean 1D contenant exactement un FALSE**.
2. Placer `B = Build Array`, agrandir à trois entrées et activer **Concatenate Inputs**. Câbler : entrée 0 = `[FALSE]`, entrée 1 = Bits, entrée 2 = `[FALSE]`. Nommer le résultat `avecBords`.
3. Placer `A = Array Size` sur Bits ; `N = Add` avec A.sortie en x et constante 1 en y.
4. Placer `F = For Loop`. N.sortie → terminal N. `avecBords` entre par un tunnel **sans indexation**. Aucun registre.
5. Dans F placer : Add, deux Index Array, deux NOT, deux AND. Câbler exactement :

Objet	Entrées	Résultat utilisé
J = Add	i → x ; 1 → y	i+1
P = Index Array	avecBords → array ; i → index	précédent
C = Index Array	avecBords → array ; J.sortie → index	courant
NP = NOT	P.element	non précédent
NC = NOT	C.element	non courant
D = AND	NP.sortie → x ; C.element → y	début de trait
E = AND	P.element → x ; NC.sortie → y	fin de trait

6. Tirer le fil `i` vers **deux tunnels de sortie distincts** de F. Clic droit sur chaque tunnel → Tunnel Mode → **Conditional**. Ces tunnels doivent construire chacun un tableau I32 1D.
7. D.sortie → petit terminal conditionnel du premier tunnel ; E.sortie → condition du deuxième tunnel.
8. Premier tableau de sortie F → S0.debuts ; deuxième → S0.fins. `error in` → S0.erreur. Relier les sorties S0 aux indicateurs.

La boucle effectue longueur(Bits)+1 tours, i=0 jusqu'à longueur(Bits) incluse. Le tableau complété a longueur(Bits)+2 éléments ; les lectures i et i+1 sont donc toutes valides. L'indice i est déjà celui de l'image d'origine : **ne pas retrancher 1**.

G3.3 Tests à réaliser

Bits	Debuts	Fins
F,T,T,F,T,F	1,4	3,5
T,T,F,T	0,3	2,4
T,T	0	2
F,F	vide	vide
vide	vide	vide

Tous ces essais ont un error out sans erreur. Le cas sans trait sera rejeté par ComputeRowsCols, pas ici. Avec erreur entrante 123 : sorties vides, même erreur 123. Sauvegarder.

G4. `ComputeCellRect.vi` — case avec bord mesuré

G4.1 Face avant

Contrôles `ligne` I32=0, `colonne` I32=0, `Lignes` instance de LignesGrille.ctl vide, `error in`. Indicateurs `Rectangle` instance RectangleCellule.ctl à zéro, `error out`. Attribuer le connecteur G1.

Le VI coupe **juste à l'intérieur des traits** *(modifié le 30.10)* : la case commence au premier pixel après le trait de gauche (ou du haut) et s'arrête avant le trait de droite (ou du bas). Pour un trait [debut,fin), son épaisseur est fin−debut ; elle sert seulement à vérifier que le trait existe. On ne modifie aucun pixel et on ne dessine pas de cadre.

- gauche = finX[colonne] ;
- droite = debutX[colonne+1] ;
- haut = finY[ligne] ;
- bas = debutY[ligne+1].

Les chiffres étant testés avant la case vide (Ed #28, #30), retirer les traits rend le test du vide robuste : une case vide est à 100 % de blanc, loin du seuil 88. L'ancien choix, garder deux pixels de trait, ne laissait que 1,5 point de marge. Les deux dimensions **du rectangle final** doivent être entre 50 et 1000 inclus. Le maximum 4096 pour les coordonnées est la garde de lecture d'image du projet. Les chiffres gardent leur taille originale 32×32 ; aucune interpolation n'est ajoutée.

G4.2 Premier filtre : ne jamais indexer hors des tableaux

1. Placer S0 Case sur error in, sorties `rectangle`, `erreur`.
2. S0.Error : rectangle constant(0,0,0,0) → rectangle ; error in → erreur.
3. S0.No Error : placer `Unbundle By Name` sur Lignes, agrandir à quatre champs exacts debutX,finX,debutY,finY.
4. Placer quatre Array Size, un par champ. Nommer les longueurs AX,BX,AY,BY.
5. Placer Subtract(AX,1), résultat n ; Quotient & Remainder(n,2), utiliser seulement remainder.
6. Placer les **neuf** comparateurs suivants, chaque ligne étant une primitive et ses deux entrées :

Test	Primitive	x	y
T1	Not Equal?	AX	BX
T2	Not Equal?	AX	AY
T3	Not Equal?	AY	BY
T4	Less?	AX	3

T5	Less?	ligne	0
T6	Less?	colonne	0
T7	Greater Or Equal?	ligne	n
T8	Greater Or Equal?	colonne	n
T9	Not Equal?	remainder	0

7. Build Array neuf entrées scalaires, T1..T9 dans cet ordre → Or Array Elements → ErreurSi.Condition. Code 6104, Message `Indices de case ou tableaux de traits invalides.`, error in = erreur entrante.
8. Placer S1 Case sur ce error out, avec sorties rectangle/erreur. S1.Error : rectangle zéro et erreur venant d'ErreurSi. Relier sorties S1 vers sorties S0.

G4.3 Dans S1.No Error : mesurer les quatre traits puis calculer

1. Add(colonne,1)→c1 ; Add(ligne,1)→r1. Huit Index Array donnent les valeurs suivantes :

Nom	Tableau	Indice
dx0	debutX	colonne
fx0	finX	colonne
dx1	debutX	c1
fx1	finX	c1
dy0	debutY	ligne
fy0	finY	ligne
dy1	debutY	r1
fy1	finY	r1

2. Quatre Subtract : eg=fx0−dx0, ed=fx1−dx1, eh=fy0−dy0, eb=fy1−dy1. Ces épaisseurs servent uniquement aux tests du point 4 *(modifié le 30.10)*. Si vous aviez construit quatre Less?/Select donnant bg,bd,bh,bb, supprimez-les.
3. gauche=fx0 ; droite=dx1 ; haut=fy0 ; bas=dy1, câblés directement *(modifié le 30.10)*. Calculer largeur=droite−gauche et hauteur=bas−haut.
4. Faire les **quatorze tests** : eg<1, ed<1, eh<1, eb<1, dx1<=fx0, dy1<=fy0, largeur<50, largeur>1000, hauteur<50, hauteur>1000, gauche<0, haut<0, droite>4096, bas>4096. Chacun utilise Less?, Less Or Equal? ou Greater? avec les deux opérandes indiquées.

5. Build Array quatorze entrées→Or Array Elements→ErreurSi.Condition ; code 6104 ; message `Traits ou rectangle invalides ; dimensions requises 50 a 1000.` ; première erreur de G4.2→error in.
6. Bundle By Name sur Rectangle constant zéro : les quatre résultats du point 3 vers gauche,haut,droite,bas. Case S2 sur la dernière erreur : No Error→rectangle calculé et erreur ; Error→rectangle zéro et erreur.
7. Remonter S2→S1→S0→indicateurs. Aucun indexage ne se fait dans une branche Error. Les calculs du point 2 sur une épaisseur invalide ne donnent jamais un rectangle utilisable, car le point 4 les refuse.

G4.4 Essais précis

Saisir debutX=[0,58,116], finX=[2,60,118], debutY=[0,53,106], finY=[2,55,108].

ligne,colonne	Rectangle (gauche,haut,droite,bas)	Résultat
0,0	2,2,58,53	largeur 56 hauteur 51, aucune erreur *(modifié le 30.10)*
1,1	60,55,116,106	largeur 56 hauteur 51, aucune erreur
2,0	0,0,0,0	erreur 6104
0,-1	0,0,0,0	erreur 6104

Raccourcir finX d'un élément : erreur 6104. Mettre finX[0]=debutX[0] : trait vide, erreur 6104. Erreur entrante 123 : rectangle nul et même erreur 123.

Pour tester la **largeur w** indépendamment : debutX=[0,w+2,2*w+4], finX=[2,w+4,2*w+6] *(modifié le 30.10)*, conserver les tableauxY ci-dessus ; appeler case 0,0. Faire w=49,50,1000,1001 : refus, succès, succès, refus. Pour tester la **hauteur h**, conserver X=[0,58,116]/[2,60,118] et utiliser debutY=[0,h+2,2*h+4], finY=[2,h+4,2*h+6]. Même séquence h=49,50,1000,1001. Ces valeurs sont des calculs de préparation du test ; n'inscrivez aucun attendu dans ComputeCellRect.

Test de bord extérieur épais : remplacer seulement debutX[0] par 0 et finX[0] par 6, avec début du trait suivant 58. La case 0,0 commence à gauche 6 et mesure 52 de large : aucun pixel de cette bordure n'est conservé *(modifié le 30.10)*. Sauvegarder après tous les essais.

G5. `ComputeRowsCols.vi` — nombre de cases et traits mesurés

G5.1 Face avant et squelette des cas

Contrôles Bitmap Boolean 2D vide, error in. Indicateurs n I32=0, Lignes LignesGrille.ctl vide, error out. Attribuer connecteur G1.

Créer S0 Case sur error in. Dans son No Error seront imbriqués successivement S1,S2,S3,S4. **Chaque Case a exactement trois sorties** : `n`, `lignes`, `erreur`. Dans le cas Error de chacune : I32 0 → n ; cluster LignesGrille dont les quatre tableaux sont vides → lignes ; erreur qui sélectionne ce Case → erreur. Dans le No Error, les trois sorties du Case intérieur rejoignent les trois sorties du Case extérieur. Seul S4.No Error fournit les résultats finaux valides. S0.sorties vont aux indicateurs.

Cette règle définit explicitement les cinq branches d'erreur. Il faut les câbler toutes ; aucun tunnel de sortie par défaut.

G5.2 S0.No Error : dimensions avant toute opération

1. Array Size(Bitmap) → tableau dimensions ; deux Index Array aux indices 0 et 1 donnent hauteur et largeur.
2. Quatre tests : hauteur<32, hauteur>4096, largeur<32, largeur>4096.
3. Build Array des quatre tests → Or Array Elements → ErreurSi.Condition ; Code 6101 ; Message `Dimensions d'image hors de 32 a 4096 pixels.` ; error in entrante → error in.
4. ErreurSi.error out → sélecteur S1 et son fil d'erreur. Bitmap entre dans S1 sans indexation. Le domaine32..4096 est notre garde pour les images, distinct des dimensions 50..1000 des cellules. Ne pas imposer hauteur=largeur ici.

G5.3 S1.No Error : trouver la position réelle du bord noir

Placer ces objets :

Repère	Objet	Configuration et fils
F1	For Loop	Bitmap entre avec auto-indexation : un tableau 1D ligne par tour ; N non câblé ; aucun registre
O1	Or Array Elements, dans F1	ligne → entrée ; sortie scalaire vers tunnel de sortie F1 **auto-indexé**
T	Transpose 2D Array	Bitmap → entrée ; sortie vers F2
F2	For Loop	transposé entre auto-indexé : une colonne d'origine par tour ; N non câblé ; aucun registre

O2	Or Array Elements, dans F2	colonne → entrée ; sortie scalaire vers tunnel de sortie F2 auto-indexé
V1	EtendueNoire.vi	F1.tableau sortie → Bits ; erreur de S1 → error in
V2	EtendueNoire.vi	F2.tableau sortie → Bits ; V1.error out → error in
A1	Add	V1.Debut → x ; 10 → y ; résultat ligneAnalyse
A2	Add	V2.Debut → x ; 10 → y ; résultat colonneAnalyse
C1	Greater Or Equal?	ligneAnalyse → x ; V1.Fin → y
C2	Greater Or Equal?	colonneAnalyse → x ; V2.Fin → y
O3	OR	C1.sortie → x ; C2.sortie → y
E2	ErreurSi.vi	O3 → Condition ; 6102 → Code ; `Les lignes d'analyse a bord+10 sont hors de la grille.` → Message ; V2.error out → error in

E2.error out → sélecteur S2 et son fil d'erreur. Passer Bitmap, ligneAnalyse et colonneAnalyse à l'intérieur de S2 par des tunnels d'entrée.

Ne pas remplacer Bitmap par le tableau transposé : la transposition ne sert qu'à calculer l'occupation des colonnes. La grille peut avoir une marge blanche ; bord+10 n'est donc pas nécessairement l'indice absolu 10. Une image entièrement blanche donne 6102 et n'atteint jamais les Index Array suivants.

G5.4 S2.No Error : extraire les axes et compter les traits

1. Deux Index Array reçoivent **Bitmap original**. Agrandir pour voir les indices ligne/colonne.
2. Premier : ligneAnalyse → index0 ; **index1 volontairement non câblé**. Sortie tableau 1D = parcours horizontal.
3. Deuxième : **index0 volontairement non câblé** ; colonneAnalyse → index1. Sortie tableau 1D = parcours vertical.
4. Placer V3,V4 = deux IntervallesNoirs.vi. Horizontal → V3.Bits ; vertical → V4.Bits ; erreur S2 → V3.error in → V4.error in via V3.error out.
5. Bundle By Name sur cluster LignesGrille vide : V3.Debuts → debutX ; V3.Fins → finX ; V4.Debuts → debutY ; V4.Fins → finY. Résultat nommé `lignesCalculees`.
6. Pour X : Array Size(V3.Debuts) et Array Size(V3.Fins)→Add ; cette somme est le nombre de transitions. Quotient & Remainder(somme,2).quotient→Subtract(quotient,1)→nombreColonnes. Pour Y : mêmes cinq nœuds avec V4.Debuts et V4.Fins→nombreLignes. Constantes 2 et 1 en I32. Cela applique littéralement transitions/2−1 (donnée p. 27). Les restes de division sont nuls puisque IntervallesNoirs apparie les débuts et fins.
7. Quotient & Remainder(nombreColonnes,2), utiliser remainder.
8. Trois tests : nombreColonnes != nombreLignes ; nombreColonnes<2 ; remainder !=0. Les relier à Build Array trois entrées → Or Array Elements → ErreurSi.Condition.

9. ErreurSi.Code=6103 ; Message=`La grille doit avoir au moins deux lignes, autant de colonnes et une taille paire.` ; V4.error out → error in.
10. ErreurSi.error out → sélecteur S3 et son fil d'erreur ; nombreColonnes → entrée n de S3 ; lignesCalculees → entrée lignes de S3.

Les débuts et fins ont déjà été appariés par IntervallesNoirs : leur nombre est le **nombre de traits**. Un début et une fin forment deux transitions : additionner leurs nombres, diviser par 2 puis retrancher 1 donne le nombre de cases. Ne divisez pas seulement le nombre de débuts par 2.

G5.5 S3.No Error : vérifier toutes les largeurs et hauteurs

1. Placer une For Loop F3 ; n → N. Aucun arrêt conditionnel ni parallélisme.
2. Ajouter **un registre à décalage de cluster d'erreur** : clic droit sur bord gauche → Add Shift Register. Depuis l'extérieur, erreur S3 → registre gauche d'initialisation. Ce fil est obligatoire.
3. Lignes entre dans F3 par tunnel **sans indexation**. Constante 0 également sans indexation si placée à l'extérieur.
4. À l'intérieur, placer R1 et R2, deux ComputeCellRect.vi.
5. R1 : ligne=0, colonne=i, Lignes=Lignes, error in=registre gauche courant.
6. R2 : ligne=i, colonne=0, Lignes=Lignes, error in=R1.error out.
7. R2.error out → registre droit. Les deux sorties Rectangle sont volontairement non câblées.
8. Après la boucle, sortie finale du registre droit → sélecteur S4 et fil d'erreur S4. n et Lignes contournent la boucle par des fils pour entrer dans S4.
9. S4.No Error : n entrant → sortie n ; Lignes entrant → sortie lignes ; erreur finale → sortie erreur. S4.Error est le cas zéro/vide/erreur défini en G5.1.
10. Relier successivement les trois sorties S4 vers S3, S3 vers S2, S2 vers S1, S1 vers S0. Enregistrer.

R1 vérifie toutes les largeurs de colonnes à la ligne 0. R2 vérifie toutes les hauteurs de lignes à la colonne 0. Toutes les combinaisons de cellules sont alors couvertes pour ces dimensions indépendantes. Après une erreur, les itérations restantes transmettent l'erreur sans refaire de travail ; les sous-VIs sont eux-mêmes protégés par leur cas Error.

G5.6 Tests des images fournies

Brancher LireImage.Bitmap sur ComputeRowsCols.Bitmap et chaîner les erreurs. Faire chaque essai séparément et noter les valeurs réellement observées.

Image fournie	Résultat géométrique attendu
Binairo_6x6.png	image 454×431 ; n=6 ; sept traits X, sept traits Y ; aucune erreur
Binairo_6x6_rect.v2.png	image 545×360 (largeur×hauteur) ; n=6 ; sept traits X, sept traits Y ; cellules rectangulaires ; aucune erreur
Binairo_8x8.png	image 610×577 ; n=8 ; neuf traits X, neuf traits Y ; aucune erreur

Binairo_4x4_Bad.png	n=4 ; cinq traits par axe ; aucune erreur géométrique : sa contradiction relève ensuite de MATLAB
Binaro_5x6_Bad.png	six lignes et cinq colonnes : erreur 6103, n=0, Lignes vide

Sur le 6×6, ComputeCellRect(0,0) doit donner gauche 6,haut 6,droite 78,bas 74, soit 72×68. Sur le 8×8 : gauche 8,haut 13,droite 80,bas 81, soit 72×68 *(modifié le 30.10)*. Ces découpages diffèrent des fichiers binaires 74×74 du professeur ; ne comparez pas leurs octets comme s'ils étaient identiques. Comparez les symboles et les matrices.

G6. Essais géométriques sans image et extraction effective

G6.1 Construire un bitmap de test sans fichier externe

1. Créer un VI temporaire : For Loop extérieure N=108 et intérieure N=118. i extérieur=ligne ; i intérieur=colonne. Sorties des deux boucles auto-indexées, aucun registre.
2. Dans la boucle interne, construire trois intervalles noirs de colonnes : [0,2), [58,60), [116,118). Pour chacun : Greater Or Equal?(colonne,debut), Less?(colonne,fin), AND des deux. OR des trois AND→traitVertical.
3. Même montage pour les lignes : [0,2), [53,55), [106,108)→traitHorizontal. OR(traitVertical,traitHorizontal) vers la sortie auto-indexée. Le résultat final est un Boolean 2D108×118.
4. Bitmap→ComputeRowsCols. Attendre n=2, debutX=[0,58,116],finX=[2,60,118],debutY=[0,53,106],finY=[2,55,108], sans erreur. Les rectangles sont ceux de G4.4. **Ce bitmap sert à la géométrie, pas à calibrer l'OCR sur toutes tailles.**
5. Grille2 lignes/4 colonnes : garder Y, mettre N intérieur 234, avec X=[0,2),[58,60),[116,118),[174,176),[232,234). Attendre 6103,n0,Lignes vide.
6. Grille3×3 : N extérieur 161, N intérieur 176 ; Y=[0,2),[53,55),[106,108),[159,161), X=[0,2),[58,60),[116,118),[174,176). Attendre 6103.
7. Tableau entièrement FALSE 108×118 : 6102. Bitmap vide : 6101. Erreur entrante 123 : 123 avec n0/Lignes vide.
8. Exécuter les huit tests 49/50/1000/1001 de G4.4. Conserver les VIs de test dans Preuves/Essais, séparés des VIs du programme.

G6.2 Extraire une case dans le futur VI de reconnaissance

1. ComputeCellRect reçoit ligne,colonne,Lignes et le cluster d'erreur courant.
2. Placer une Case sur son error out. Dans Error : pas d'extraction, pas d'écriture, pas d'appel OCR ; propager l'erreur.
3. Dans No Error : Unbundle By Name(Rectangle) donne gauche,haut,droite,bas.
4. Deux Subtract : droite−gauche → largeur ; bas−haut → hauteur.
5. Array Subset reçoit Bitmap original 2D ; régler pour deux dimensions : index0=haut, length0=hauteur, index1=gauche, length1=largeur.
6. Vérifier Array Size du résultat : index0 doit donner hauteur, index1 largeur. Ce tableau reste Boolean 2D avec TRUE noir.
7. Le sous-VI d'écriture du chapitre suivant le convertira en U8 et écrira hauteur puis largeur, dans l'ordre d'Array Size *(modifié le 30.10)*. **Ne pas transposer pour corriger l'ordre de l'en-tête.**

Le calcul suppose une grille alignée sur les axes et des lignes d'analyse à bord+10 qui ne traversent pas les chiffres, comme dans l'énoncé. Les données fournies et les cas limites ci-dessus doivent tous être essayés sur la VM. Une flèche exécutable et des résultats attendus dans un document ne remplacent pas ces essais.

Mise a jour des sources et du PDF

Les fonctions DirectValues, CheckValidMove, CheckVectorOk, CheckVectorUniqueOk et printGrid sont locales a SolveBinairo.m, comme dans le modele fourni. Supprimez leurs anciens fichiers .m du dossier Execution. Elles ne s'appellent plus directement depuis la fenetre de commande. Le script genere utilise disp(B) pour afficher la matrice ; remplacez aussi printGrid(B) par disp(B) dans une ancienne constante N6.

Le dessin utilise axis ij, axis equal et axis off. Le PDF utilise les reglages papier par defaut de MATLAB ; la date provient de char(datetime('now')) et depend de la langue et des preferences de l'installation. Verifiez le PDF natif : grille entiere, cases carrees, nom et date lisibles, couleurs et message d'erreur. Ce nouveau rendu n'a pas ete execute ici.

Apres une erreur, le C vide CellValue.txt lorsqu'il le peut (fichier inscriptible). Un fichier protege ou un dossier de ce nom ne peut pas etre vide : le code de retour reste obligatoire pour decider si le resultat est utilisable. *(modifié le 30.10)* Une erreur d'ecriture apres ouverture peut encore laisser un resultat incomplet.

Contrôles de remise après l’addendum du 7 octobre

Rédigez la documentation courte après les essais sur la VM EPFL, suivant le chapitre 21. Relevez alors la version Windows, l'identification de la VM, le compilateur et sa version, MATLAB et sa version, ainsi que LabVIEW et sa version. Documentez la configuration dans laquelle le projet complet fonctionne ; ne recopiez pas les versions de l'exemple Billard du professeur.

Le C accepte une cellule rectangulaire dont chaque dimension est dans 50..1000 et qui contient exactement largeur×hauteur octets 0/1. Une grille Binairo, en revanche, doit avoir le même nombre pair de lignes et de colonnes. Les coordonnées et seuils restent contrôlés avant leur utilisation. Les codes−2/−3 cités en cours étaient des exemples, et non une numérotation imposée. Les codes actuellement implémentés sont 0,2,3,4,5,6,7 (7 : 0 et 1 admissibles à égalité, Ed #7) *(modifié le 30.10)* ; leur signification est précisée dans reconnaissance.h et les messages d'erreur. Toute erreur non nulle interdit de lire l'ancien résultat.

Pendant les essais natifs, vérifier aussi le refus d'un fichier tronqué, d'un octet supplémentaire, d'une dimension négative encodée en U32 et d'une dimension 10000. Conserver le message complet, pas seulement le numéro.

Annexe H - Techniques et références ME-213 / CS-119a

Ce tableau relie les **techniques des sources C/MATLAB et des diagrammes décrits** aux cours fournis. Le projet combine ces techniques pour résoudre un nouvel exercice : il ne prétend pas que chaque ligne a été recopiée d’une diapositive. Les pages sont physiques, à partir de 1. Les fichiers originaux des cours ne sont pas redistribués dans le dépôt.

H1 C

Technique dans le projet	Référence fournie	Utilisation
Conditions, boucles, tableaux, fonctions, pointeurs, chaînes terminées par NUL	CS-119a `cours2.pdf` à `cours7.pdf`, exemples `while_number_of_digits.c`, `strings_argc_argv.c`, `pointers_and_arrays.c` ; ME-213 C1	parcours des arguments et des pixels, validation, fonctions auxiliaires
Conversion d'un entier par les chiffres, multiplication par 10 et addition	opérations/arithmetic et boucles des cours précédents ; `while_number_of_digits.c` pour le parcours des chiffres	ConvertirSeuil : refus des fractions et bornes 1..99, sans parseur externe
Structures et pointeurs vers structures	CS-119a `cours8.pdf`, p. 18–29 ; `structs_and_ptrs.c`	CelluleOCR, SeuilsOCR, ResultatOCR
malloc/free, allocation selon la taille connue	CS-119a `cours7.pdf`, p. 10–11 et 29 ; `dynamic_array.c` ; ME-213 C5	pixels et chemins ; aucun VLA
fopen/fread/fgetc/fprintf/ferror/fclose	ME-213 `C3.PPI_C_Fichiers.26.r1.pdf`, p. 13–26 ; CS-119a `cours11.pdf` et `cours12.pdf`	lecture binaire groupée, détection des troncatures/excès, texte de sortie, erreurs
fflush	CS-119a `cours12.pdf`, p. 12	détecter l'échec du vidage du tampon de sortie
strlen/strcmp/memcpy	CS-119a `cours12.pdf`, p. 21,23,25–26 ; `strings_strlen.c`, `strings_strcmp.c`	construire le chemin voisin CellValue.txt sans modifier l'entrée
Types entiers de largeur fixée, octets et ordre des octets	ME-213 C3, p. 27 et 30–34 ; donnée Binairo, p. 20–22	uint32_t, décodage little-endian
Décalages et masques binaires	ME-213 `C2.PPI_C_RepresentationNombre+BitWiseOp.26.r1.pdf`, p. 13–16	GetDigitBitmapBit et lecture de l'en-tête
static, const, durée de vie	ME-213 `C5.PPI_C_IntroMemoryMgmt.26.r1.pdf`, p. 8–9 ; rappels C	nom constant de sortie

goto uniquement vers le nettoyage final	ME-213 `C4.PPI_C_SoftwareEngineering.26.r1.pdf`, p. 21–23	un chemin de fermeture/libération en cas d’erreur
Balayage des deux modèles 32×32, score/marge puis test du vide (Ed #28, #30)	donnée Binairo 2026.2, p. 7–18	algorithme expressément demandé

La mise à jour des seuils utilise uniquement caractères, entiers, if/while et arithmétique. Le contrôle après chaque chiffre empêche le débordement de l’accumulation. Les structures et interfaces existantes restent identiques.

H2 MATLAB

Technique/fonction dans les sept fichiers	Référence fournie	Utilisation
Matrices, produits, transposition, indexation, masques logiques, concaténation	`M1.PPI_Matlab_I.26.r1.pdf`, p. 15–29	comptes, répétition d'une ligne/colonne, comparaison simultanée et tracé sans boucles
find, ind2sub, isnan, sum, any, isequal	`Exo.Proj.Binairo.Matlab.2.pdf`, p. 1 ; M3 p. 12 pour ind2sub	fonctions explicitement proposées pour ce projet
if, fonctions locales, arguments/retours, isa	`M2.PPI_Matlab_II.26.r1.pdf`, p. 27–41	validation et découpage en fonctions
Récursion et retour en arrière	donnée Binairo p. 29 et 33–34 ; exercice de récursion ; CS-119a `cours13.pdf`, p. 5–7 et 18–21, `cours14.pdf`	essais 0 puis 1 et reprise de la grille de base
Diviser un parcours en deux parties récursives	CS-119a théorie `icc_cours3_handout.pdf`, p. 12–24 (dichotomie/tri fusion)	ParcourirCases conserve l'ordre mais limite la profondeur du parcours ; aucune bibliothèque de recherche
numel et isempty	`Demo_matlab.zip`, `Demo_Matlab_4_8_Integral_SimpsonRec.m` ; pour isempty aussi `Demo_Matlab_4_4c_DemoAntOutsideThePlate.m`	taille et absence d'éléments
floor	`Demo_matlab.zip`, `Demo_Matlab_4_x_CellArray_SegmentGetCentralIdx.m`	milieu entier d'un intervalle
Cell arrays et concaténation de cellules	M3 p. 15–19	découper le nom long en lignes de texte
meshgrid	M3 p. 28–30	coordonnées de toutes les cases
figure, axes, handles et propriétés graphiques	M2 p. 42–50 ; `Demo_Matlab_2_5_PlotRef.m`	placement, couleurs, tailles et propriétés du dessin
line	`Demo_Matlab_2_1_edit_debug.m`, et démonstrations d'animation de ressort dans Demo_matlab.zip	traits horizontaux et verticaux

text, axis, datetime, num2str, print	`Exo.Proj.Binairo.Matlab.1.pdf`, p. 2	fonctions explicitement proposées pour le PDF ; char convertit la date en texte
close, drawnow	`Demo_Matlab_4_15a_ODE45_Unforced_DampedSpring_Animation.m`	rafraîchir/fermer la figure sans toucher aux autres figures
fprintf et error	M2 p. 32 ; M3 p. 51 ; squelette SolveBinairoHeader	messages et remontée des erreurs
uiopen	addendum du 7 octobre, p. 14	ouvrir le PDF si demandé

Le produit des masques 0/1 dans GrilleBinairoValide n'introduit pas une nouvelle bibliothèque : il applique le produit matriciel enseigné pour compter les positions égales. Une somme égale à n implique deux lignes complètes identiques. Les récursions, opérations matricielles et signatures du squelette sont conservées. Aucun arrayfun/cellfun, classe, toolbox d'optimisation, solveur externe ou boucle explicite n'est ajouté.

H3 LabVIEW

Construction décrite dans la recette	Référence fournie
Types, conversions, tableaux/clusters, For/Case, Select	`LV1.PPI_LabVIEW_intro1.26.r1.pdf`, p. 28–47,59–61,71 et 75–76
Registres à décalage et propagation d'erreurs	`LV2.PPI_LabVIEW_intro2_SR_Files.26.r1.pdf`, p. 3,8–10,18–20 ; exercice Loops/TunnelMode/ShiftRegister
Chaînes, String Subset, Search and Replace, Format Into String, Scan From String	LV2 p. 45–50
Fichiers texte/binaires, ouvrir/écrire/fermer	LV2 p. 24–31 ; exercice Binairo LabVIEW 1, p. 9–14
System Exec et gestion de son retour	LV2 p. 51 ; exercice Binairo LabVIEW 1 p. 13–14 ; LabVIEW 2 p. 2–7
Lecture PNG, Picture to Pixmap, Unflatten Pixmap, Index Array, Array Subset	exercice Binairo LabVIEW 1 p. 3–8
Chemins relatifs au VI, Build Path	même exercice p. 10–11
Compter transitions et traiter les traits aux bords	donnée p. 27 ; exercice LabVIEW 1 p. 15–16
Nouveau calcul du rectangle	indexation des traits, soustractions, comparaisons et Select des cours ci-dessus ; pas de morphologie ou de traitement d'image ajouté
Parseur OCR sans expression régulière	Scan From String puis Format Into String et égalité de chaînes, LV2 p. 47–48
Génération de solve.m	LabVIEW 2 p. 2–5, chaînes/formats de LV2
Suppression d'un ancien résultat, contrôle d'existence/type et d'erreur	fichiers/chemins/erreurs de LV2 ; détails des bornes via Context Help, comme demandé dans l'exercice

Les détails des **bornes et options des fonctions natives** se vérifient avec Context Help et la documentation NI, comme le demandent les exercices. Ce sont des paramètres d'utilisation des fonctions du cours, pas un nouveau paradigme. Les fonctions de lecture d'image et les opérations de fichier restent natives. La recette ne demande ni Python, ni .NET, ni ActiveX, ni IMAQ, ni DLL externe, ni expressions régulières. Les commandes du compilateur et les fichiers projet sont des moyens de construction, pas du code d'algorithme à étudier.